

24/10/24

Language Processor

- compiler → compile kore assembly code e translate kore
- interpreter → like python.
- assembler (assembly code to machine code e convert kore)

Compiler

eg → given a 4 line code

```
{ int a;  
  a = 10  
  int b  
  b = a
```

→ ekkare error msg msg dei
compiler pura code ~~the~~ nei, niye compile
kore jaja error ache bla bla shob dei
→ interpreter → line by line

ek line kore execute hoy
2 error pelei dei

compiler needs a

storage (shob error msg
rakhar jonno).
has error handler.

Q) diff between compiler & interpreter.

compiler → error peleo next line execute hoy.

HLL to low level lang:

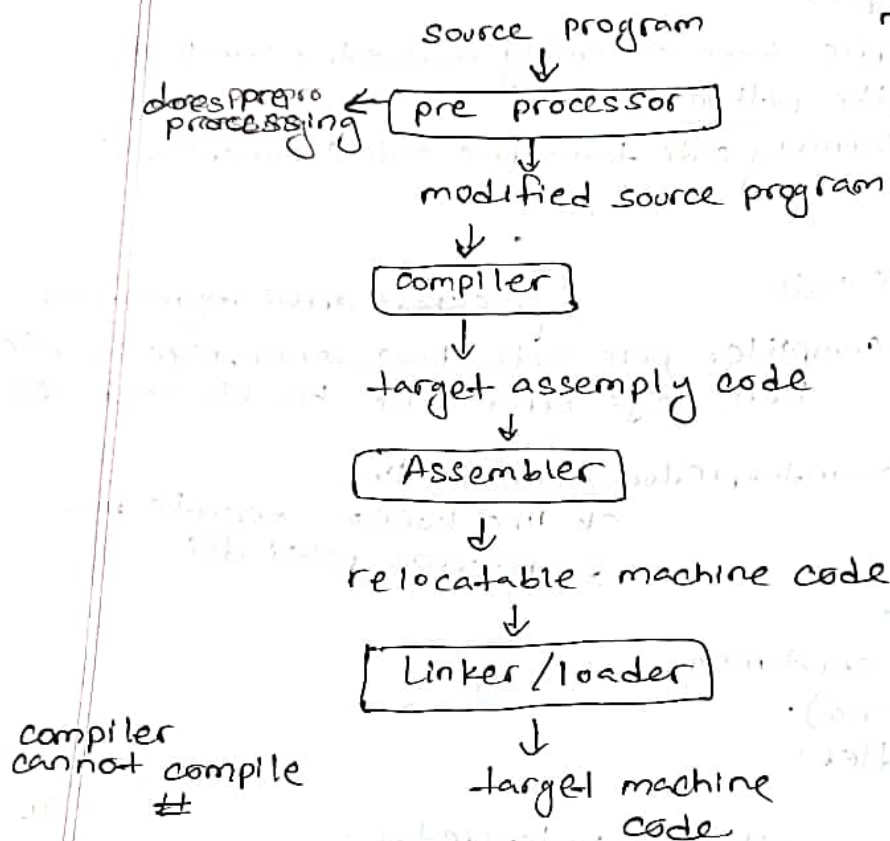
source Program

↓
pre processor

Read-Book

cont

written code to machine code overall process.



compiler cannot compile #

processor → removes all # from code

pi converts to 3.1416
operator conversion

pre processor defines
i++
i = i + 1

scope. h eita to remove
hoye jay, tai
ei header file ta
load kore rakhe
putra.

compiler pi k ekta
variable hishabe count korbe.
pre processor pi er jaygay
3.1416 boshay.

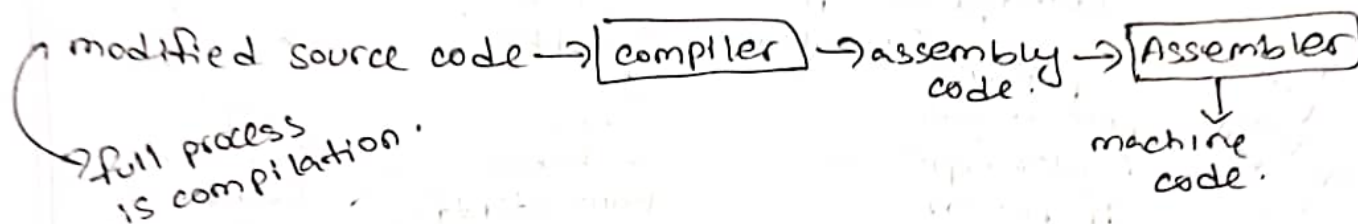
* after all
these,
we get
modified
source code
that goes to
compiler

compiler $\rightarrow$ compiles & converts to assembly code.

Hybrid compiler

interpreter + compiler or combination used in java
eg. java jdk/have give

rec 12



6 stages inside the compiler

modified source code

$\downarrow$ lexical analysis $\rightarrow$ generates token for source code.

$\downarrow$ token generation

Syntax analysis

$\downarrow$ Parse tree

Semantic Analysis (if parse tree)

$\downarrow$ parse tree

Intermediate code (if parse tree)
generation

Three address code

$\rightarrow$ generates a 3 address code.

$\downarrow$ code optimization. # no three pass based on rules (grammar)

$\downarrow$ optimized three address code.

code generation.

(target machine code)

code optimization

$$t_1 = t_2 + t_3$$

$$a = t_1$$

optimized :

$$a = t_2 + t_3$$

$$t_1 = t_2 + t_3$$

$$t_5 = t_1 \times t_4$$

$$t_5 = (t_2 + t_3) \times t_4$$

4 add code

$\therefore$ optimization not possible, already optimized.

eg) modified source code

int a; int b; ; → line end.
 ↳ Identifier (var, array, func, class name)

lexical analysis phase 1
 token generate hoy.

int → INT
 ; → SEMICOLON

Ident Identifier hole token ID set korbo.

{ identifier } → ID (always last notation e trakhbo)

↓
 ei pattern pele we goto regular expression.
 If regular expression er shathe match kore, we generate token ID

or else
 2 shobar jonno identifier token generate kore.

a b
 ↓
 ID

Suppose aro Identifier a ache.
 2 I am in Symm Semantic phase tokhon if I need to check values, how do I know kanta A 2 kanta B er ID, coz shobe to ID.

int → INT
 ; → SEMICOLON

{ identifier } → ID
 ↳ [a-z][A-Z]*

not a pattern its a notation (not defined)

int a;
 int b;
 a → not a → token

∴ we make a table.
 Symbol table

INT ID SEMICOLON

INT ID SEMICOLON (int b;)

↓
 2 shob got shob pattern check korbe.

Syntax analysis $\rightarrow$ Input (stream of tokens)

2 ID tokens, how to distinguish?

$\therefore$ we store it in Symbol table.

every phase has access to symbol table

Lexical analysis
syntax "
semantic ")
blabla.

data structure
identifier tokens
store store kardo.

(int symbol table e rakho).

int ekshabhe bhai
it is a data type.

~~int~~
INT ID semicolon } total 6 lexims.
int ID " } ID er lexims
the stream of token
dekhe bhai.

{ identifier } $\rightarrow$ (ID, index)

$\downarrow$
symbol table er index e oi token ta use

int a; $\rightarrow$ INT (ID, 0) SEMICOLON

int b; INT (ID, 1) SEMICOLON.

Done token generation

input for syntax analysis.

0	a
1	b

Symbol table

CFG $\leftarrow$ { INT (ID, 0) SEMICOLON
INT (ID, 1) " }

context
free
grammar

Syntax analysis checks if the syntax is OK.
using context free grammar
& grammar diye parse tree
bani,

$\downarrow$
match kore

a=10
 $\hookrightarrow$ in symbol
table

a=10

b=20, c=30

a = b + c

$\langle ID, 0 \rangle = \langle ID, 1 \rangle + \langle ID, 2 \rangle$

int $\rightarrow$ INT

; $\rightarrow$ SEMICOLON

= $\Rightarrow$ =

+ $\rightarrow$ +

0	a, 10, int
1	b, 20, int
2	c, 30, int

int kina

int hole

int + int
kora por
store
hobe.

— + —

eikhaner jai

2 ta val add

hobe,

ei 2tai ki int
kina (from symbol
table)

If yes, the

Addition possible.

⊆

~~a + b~~ a = b + c

t1 = t2 + t3

a = t1

a = t2 + t3 (3 add code)

Analysis phase → token generation, error, syntax error, semantic error check.
3 add code generation.

↳ lexical, syntax, semantic & intermediate code generation

Synthesis → analysis phase & 3 add code to ja peyechi
oigula k target code e & newa.
↓
machine code.

Analysis phase → { lexical
syntax
semantic
Intermediate code generation
3 add code.

lexical analysis phase e jodi kono pattern
na mile, not even identifier,
tothon it goes to panic mode. (kono defined pattern
er shathe match
koreni)

while

while
↑↑↑↑↑

→ ID to somehow
matched
(no error in lexical
phase).

panic mode → ek ekta letter bad dei & milay
to assume the error.

↓
to fix na hole they give a lexical error.

if lexical
phase e error

gonno kono error
ashenai, it means

Identifier token generate

hoise → eita dhora porbe in syntactic
& phase.

— space } no ~~token~~ token

Syntactic error

→ misplaced punctuation

int a ; → Suppose dei na,
but they expect this
; token
∴ error dibe.

→ incorrect use of keywords.

while ()
suppose ← while
lexical token generate hoye
gese.
ID
condition
thakar katha.

ID ()
↓
eikhan
kono
condition
thakar katha na,
grammar er shathe
match korena
∴ Syntax error.

→ mismatched paranthesis

Suppose a nested loop

not closed { }
a token already generated.

only generates
lexical token
phase e
grammar hoy na

Syntax phase
e grammar
hoy only

expect a } ∴ grammar
match
kore na.

same lexicon or token

→ incorrectly structured statements

if (x == 10) {
 ↓
 missing

if (x=10)
 Wrong

if (ID ==)
 ↓
 missing

grammar

==

↓
match

match korena:

Semantic error → in semantic analysis phase.

type checking.

→ undeclared variable

a = 10

↓

← ID



Symbol Table

→ Symbol table error info na!

→ type checking, ~~mis~~ mismatching
type = m

string a; | int a = 'b'

a = 9

X

X

type mismatching.

→ misuse of operators.

+ - → context free grammar ~~on~~ onujay use korena:

→ logical errors

~~z = x * y~~ z = x * y
 ✓

but written

in

z * y = z

X

new variable add howa mane
ID token generate hooche.

yylex() = int x;
int a;

→ loop
ek bar por ek bar.
ashe
puran da haray
jay.
∴ slotte ba kotte
hole agei kotte
hole
in symbol
table.

total
no.
of tokens
lexims

1 2 3 4 6 7 8
int sum(x, y) {
5
9 return x + y;
10 11 12 13
14 }

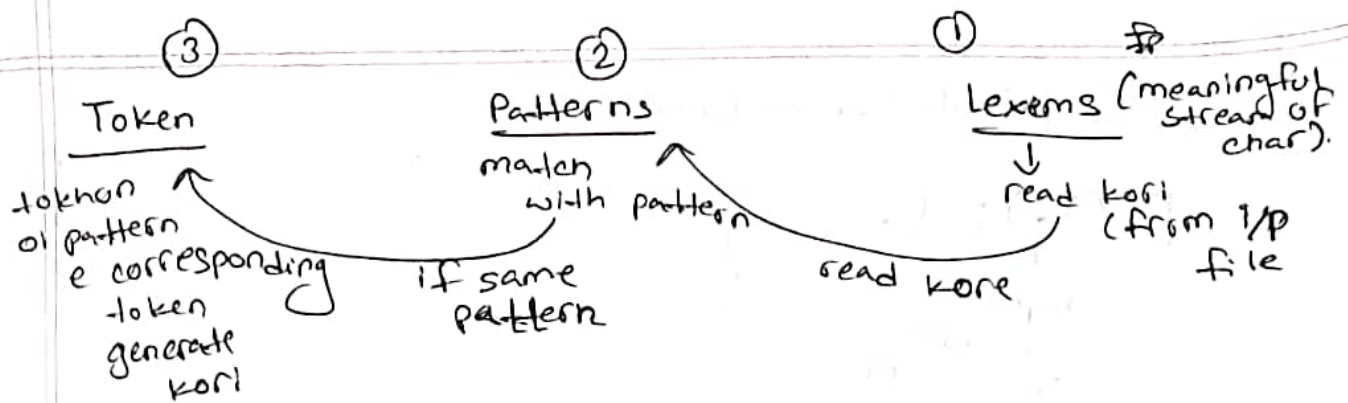
not mention
same lexim
e same token

lexim → meaning ful
stream of
characters.

~~IF~~ IF { IF { }

IF ← if { }

Individual IF
tokens generate
hooche
but same IF token
i.e IF



RE → identifier, no. match korar jonno

$a^* \rightarrow \epsilon, a, aa, aaa, \dots$
 ↓
 kleen closure.

$a^+ \rightarrow a, aa, aaa, \dots$
 ← at least 1 bar a thakbe

$ab \rightarrow ab$
 not ba

$a|b \rightarrow a, b$
 a or b

$[abc] \rightarrow$
 choose either a or b or c
 jekono ekta
 (at a time 2 ta select hocche na)

$[a-z] \rightarrow$ anything from this range but jekono ekta select hobe.

$a^2 (a\{2\}) \rightarrow aa$
 a exactly 2 bar repeat hobe

$(a\{2,3\})$
 ↓
 a min 2 bar 3 bar theke beshi repeat hobe.

$[]$
 X bari thakle means bhitore ja ache 1 ta choose hobe.

~~$(a\{2,3\})$~~

\ → dewar mane oi char tai hobe

[a-z] → a to z er moddhe jai konor ekda

[a(\-)z] → a - z
↓ ↓
hyphen e char
boshbe now
(no range)
\' er por ja ache
oi char tai hobe

\ @
\\
↓
literal
eitai boshbe
in REGEX

a(\|b) → a|b
↓
char

~~+(abc)~~

bracket dewa lage (|-)
like

{ color
| | | | \
colour
↓

u hotoo pare
nao hotoo pare & hole 1 bar hobe

colour?
u

u er por e ques mark matk.

? er
age ja ache
she repeat
hotoo pare
nao hotoo pare
& holeo 1 bar
hobe.

colo(aub)?

Syntax Analysis

31/10/24

↑
input stream of tokens

6 step & symbol table
& error handler
0. connected

Also called parsing phase (Parsing)

* stream of tokens $\rightarrow$ Syntax Analysis $\rightarrow$ Parse Tree

@ CFG er basis e stream of tokens er jonno parse tree generate korchhi.

Syntax ~~ana~~ analyser / parser.

Parse tree banate para means my stream of tokens correct.

Parsing

int x;

INT ID semicolon.

$\rightarrow$ left to right e parsing hoy.

int a;

int b;

int c;

Tokens er parsing hoy not code

= parsing always left to right hoy.

LL $\rightarrow$ left to right (regular parsing), Left most derivation.

LR $\rightarrow$ left to right, right most derivation.

Types of Parser (3)

$\rightarrow$ Universal (can be ignored)

$\rightarrow$ top down

$\rightarrow$ bottom up

— root

top down

root to leaf

— — leaf

bottom up

leaf to root

CFG

1. Terminals
2. Non-terminals
3. Start state (root node)
4. Productions

start state,
not always started

production
rule

4 production
rules

$\text{start} \rightarrow \text{id} = \text{expression}$
 $\text{expression} \rightarrow \text{expression} + \text{term}$
 $\text{expression} \rightarrow \text{term}$
 $\text{term} \rightarrow \text{term} * \text{factor}$
 $\text{term} \rightarrow \text{factor}$
 $\text{factor} \rightarrow \text{id}$
 $\text{factor} \rightarrow \text{num}$

Productions

production $\rightarrow$ production
head = body
either $\rightarrow$ / $=$

formal way,
start state
given above

how to figure start state?

is id / num in the right or left of arrow?

arrow & left side & that's production head.

& arrow & right side & that's production body.

num is reduced in factor
 $\downarrow$
not a production
head anywhere.

num & id is not ~~reduced~~ produce the production head $\therefore$ they are leaf nodes.

now,

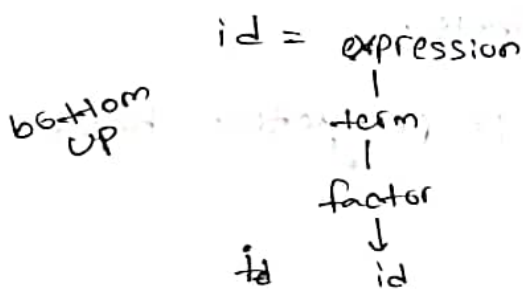
if ~~it is~~ factor ~~get~~ is reduced anywhere?
if not, that means factor is the
start state.

Start state
middle to the
part to
confuse

Since factor is reduced in term, factor is not the start.
Similarly term is also not the start.
& " expression " " " " " "

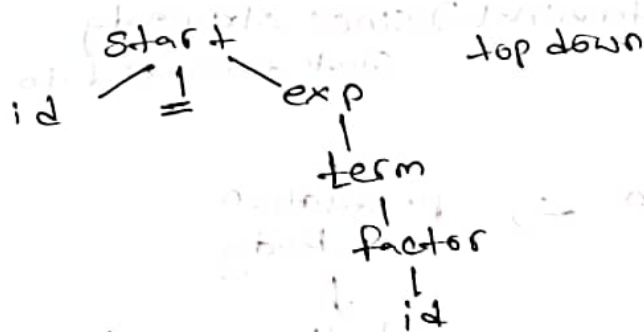
eg { $exp \rightarrow exp + term$
 $exp \rightarrow term$
 $term \rightarrow term \times factor$ } exp is the starting state here
 & exp can be the production body.

if $exp \rightarrow exp$ X



$exp \rightarrow exp + term$
 $exp \rightarrow term$
 $start \rightarrow id = exp$
 $term \rightarrow factor$
 $factor \rightarrow id$
 $factor \rightarrow num$

top to bottom, if we only successfully find the starting state.



Terminals $\rightarrow$ ① direct tokens

② - they are never in production head.

Terminals are leaf nodes.

$exp \rightarrow term$

$term \rightarrow * factor$

$factor \rightarrow id$

$factor \rightarrow num$

id/num can never be terminals.

$term/exp$ can be terminals.

Non-terminals $\rightarrow$ Always in production head.

$term \rightarrow term * factor$

terminals $\rightarrow = * id, num$

non-terminals $\rightarrow exp, term, factor$

Start also non-terminal $\rightarrow$ since already started, no need to state

Production head

PRO



non-terminals



Production

Body



both terminals &

non-terminals.

eg: $term = term * factor$

$[a-z]^* [A-Z]^+ \rightarrow$ regular definition
 $[small]^* [capital]^+$

$small \rightarrow [a-z]$

$capital \rightarrow [A-Z]$

$digit \rightarrow [0-9]$

$[0-9]^*$
 $[digit]^*$

Ambiguous $\rightarrow$ 1 ~~an~~ inp string. er jonne, with.

Same CFG, 1 tar ~~ph~~

beshi parse tree banate

parsi

Unambiguous $\rightarrow$ vice versa.



Bottom up Parsing

1st $E \rightarrow E + T \mid T$

2nd $T \rightarrow T * F \mid F$

3rd $F \rightarrow (E) \mid id$

input string ; $id^1 * id^2$

from lexical analysis part:
find if this i/p string is grammatically correct.

$() \rightarrow$ ekta o match kore lagbe.

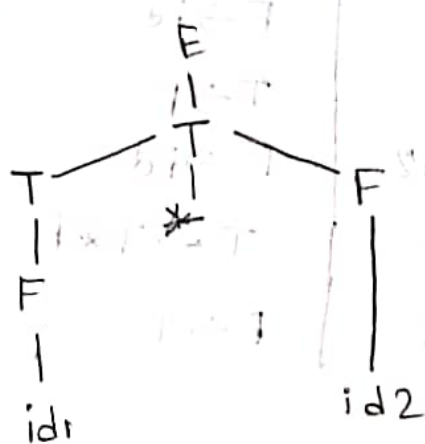
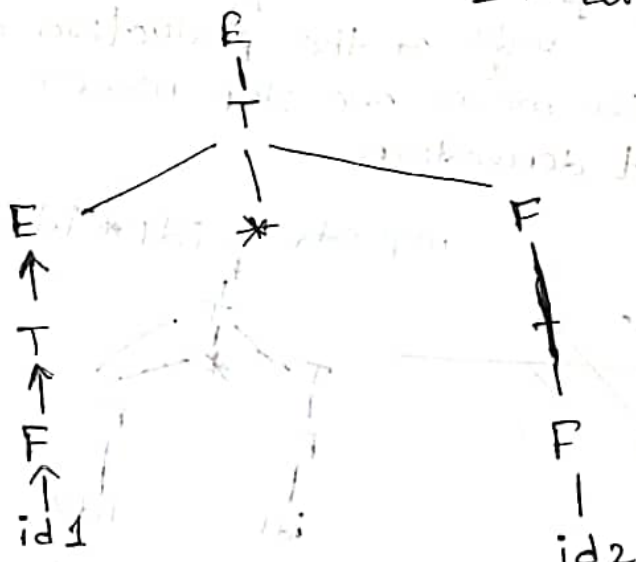
1 ta kore token jabe.

$E \rightarrow T$

2nd {

from 3rd {

wrong



how does comp know this?

we need a parsing table

✓

reduction

$$F \rightarrow id$$



id k F e reduce korchhi...

Handle :- a substring that matches the body of the production whose reduction means one step reverse of a right-most derivation

① id1 k reduce kore
F e nichhi
≠ $F \rightarrow id$
k reduce hoyeeche
2 kiche reduce hoyeeche

inp str $\rightarrow id1 * id2$



* handle
↓
k reduce hoyeeche

Right Sentential Form	Handle	Reducing Production
currently $id1 * id2$	$id1$	$F \rightarrow id$
$id1$ switched with F	F	$T \rightarrow F$
$\because id1$ reduced in F	$id2$	$F \rightarrow id$
$T * F$	$T * F$	$T \rightarrow T * F$
T	T	$E \rightarrow T$
E		

if show this also if they ask to explain/ describe Handle

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Input string: $id1 * id2$

Shift-Reduce Parse

↓
ekta ilp str is grammatically correct kina.

→ parse table missing!

4 operations

- ① Shift
- ② Reduce
- ③ Accept
- ④ Error

id2
*
T
\$

Input

Stack

Input Buffer

Action

Initially \$

\$ id1

* id2 \$

Reduce by $F \rightarrow id$

\$ F

* id2 \$

Reduced by $T \rightarrow F$

\$ T

* id2 \$

Shift

\$ T *

id2 \$

Shift

\$ T * id2

\$

Reduce by $F \rightarrow id$

\$ T * F

\$ \$

Reduce by $T \rightarrow T * F$

\$ T

\$

Reduced by $E \rightarrow T$

\$ E

\$

Accept.

→ jei stream of token ashse oigula eikhane thake.

jokhon bolbo shift, token input buffer er to e j ache, she stack e jabe.

Reduce bolle reduce korbo & kon rule diye reduce korbo bola thakbe.

Stack e \$ means stack e kichu nai

Stack er jei portion production rule (Action) er shathe mile, oitu ke reduce kori

TxF k T te reduce korchhi

7/11/2024

Left to right
Rightmost deriv
LR → tot bottom up
parsing

LR(0) Automaton

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

check if this grammar is LR grammar or not
OR for parse table down, check if the stream of token is correct

automation
core parse
table
build kore ho.

LR(0)

LR(k), k=1 means 1 ta token loop ahead korch

$id * id$

LR → k=1
OR SLR → SLR(1)

LR grammar/
SLR grammar
check kori
with parse
table.

Step 1 Augmentation:
starting state e extra
production rule lekhbo.

$E' \rightarrow E$
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

Step 2: Make item state.

start augmentation part

$E' \rightarrow \cdot E$

eg → $T \rightarrow T \cdot * F$
means T dekheche
1 ta slot kore dot
shift hobe.

• indicates
production
rule er
koto tuku
dekheche.
eg → always
in produc-
tion
body

$E' \rightarrow E \cdot E$
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$

• er por jache ta only
dekhbe the → non-
terminals. end e • ta
oitar por ja last e ja
ache dao dekhbe chole jabe.

* terminal

$E' \rightarrow \cdot E$ ek open
open korbe
 $E \rightarrow \cdot E + T$ E e bhiton
 $E \rightarrow \cdot T$ ja ache dao
 $T \rightarrow \cdot T * F$ dekhbe

• er por
non-terminal
thakle.

$E' \rightarrow \cdot E$ I_0
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

1st e E dkhiam
 2 open korbo

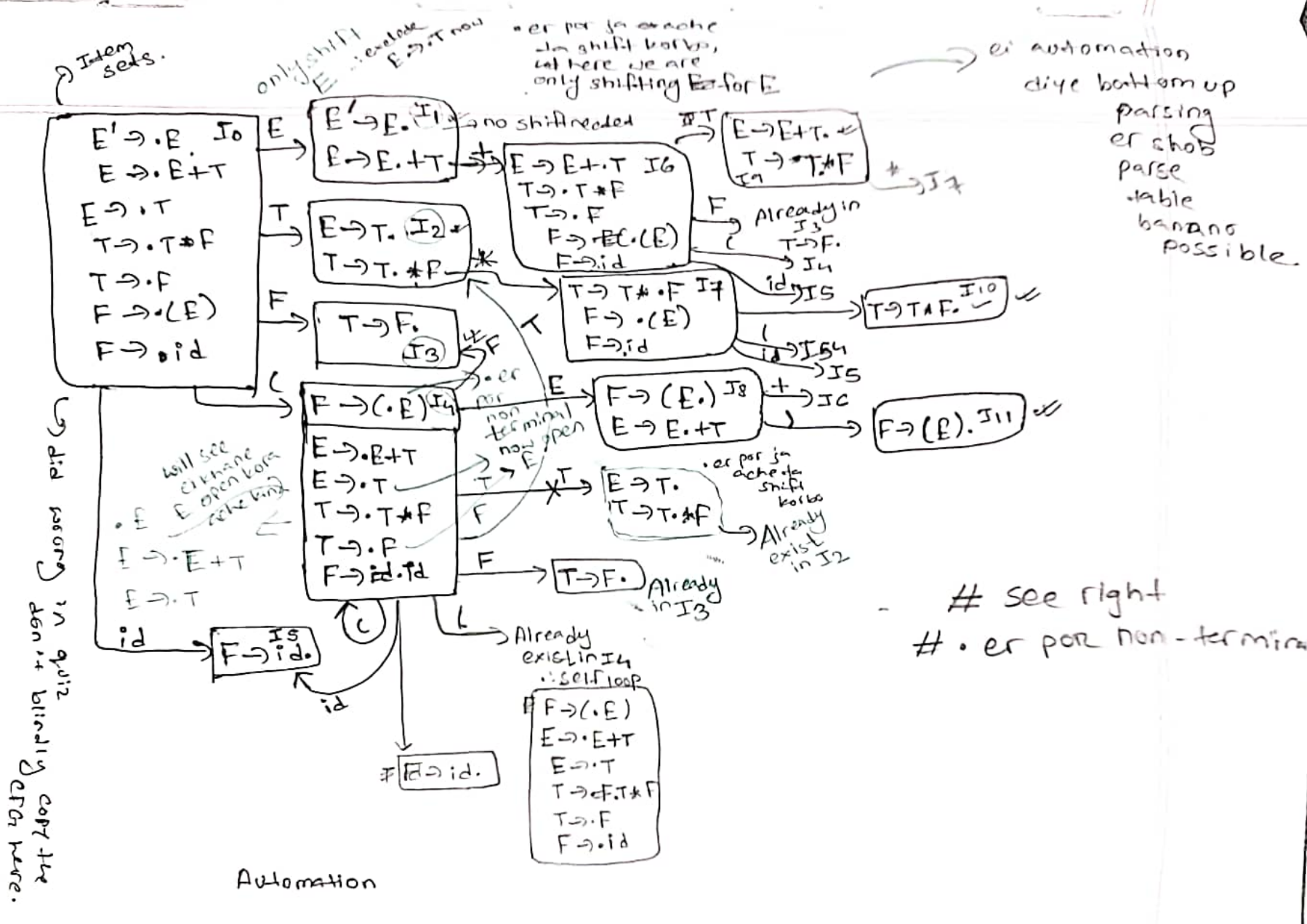
T now seen
 2 opened

do not open
 coz $\cdot$ or parz (3
 don't open
 E

now shift the
 dot 1 slot
 shift E

$E' \rightarrow E \cdot$
 $E \rightarrow E \cdot$

$\downarrow E$



conflict in parse table
 jato gula item ache I-I here
 R2 E → T reduce to F.
 parsing korbo. cos stack e 2 dekhnese S1/R2 both same for \$ er jonno or reduce hobe

Recording since beginning

LR Parse table (From automation)

9/11/24

jato gula item ache I-I here
 R2 E → T reduce to F.

State	id	+	*	(	)	\$	Go to part			
Action							E	T	F	
0				S4			1	2	3	
1		Sc				Accept				
2	R2	R2	E → T	R2	R2	R2				
3	R4	R4	R4	R4	R4	R4				
4	S5			S4			8	2	3	
5	R6	R6	R6	R6	R6	R6				
6	S5			S4				9	3	
7	S5			S4					10	
8		S6			S11					
9	R1	R1	S7/R1	R1	R1	R1				
10	R3	R3	R3	R3	R3	R3				
11	R5	R5	R5	R5	R5	R5				

forget the augmented state in action part
 Action → only terminals.
 terminals
 ↓
 never in production head
 they are direct tokens.

E → E'X
 E → E + T / T
 T → T * F / F
 F → (E) / id

0 → id → IS
 shift to S (S5)
 → S4

not other rules.

\$ → Accept likhbo when jei item e augmentation
 er dat last e thake E' → E. ✓ E' → .EX

Go to part
 ↓
 jato gula non-terminals including the start state.

Go to part e shift / reduce korina direct state no.

Reduce E → E + T (this should also be in parse table).
 → R1

Reduce rules

Reduce → 0 no. item e if a rule is fully finished like F → id. ✓ then parse table e shob gula terminal ei Reduce hobe.

- E → E + T ①
- E → T ②
- T → T * F ③
- T → F ④
- F → (E) ⑤
- F → id ⑥

Since conflict in parse table, ∴ The grammar is not LR grammar

without augmented state

2 types of conflict

→ shift reduce conflict

→ Reduce Reduce conflict
eg R/R_3

diff from LR Reduce shob gular sonno likhlabm, but now, Reduce only er Rule er production head er follow set e ja ja ache shesgular under e Reduce likhbo.

SLR Parse table

① first set → ignore the augmentation er Rule.
↓ for non-terminals only

nicher theke upore jabo of gramm.

First (E) = { (, id }
First (T) = { (, id }
First (F) = { (, id }

Rules

$E \rightarrow E + T \mid T$ ①
 $T \rightarrow T * F \mid F$ ②
 $F \rightarrow (E) \mid id$ ③

suppose F er jonno first ber karchi, kon kon production rule er production head e F ache.

$F \rightarrow (E)$ terminal directly first set e rakhbo.
 $F \rightarrow id$

$T \rightarrow T * F$

$T \rightarrow F$ non-terminal. ∴ first of this non-terminal.

∴ first of F
first (T) = first (T) ignore.

always first item tai nibo.

~~first (E) = E + T~~ $E \rightarrow E + T \Rightarrow E = \text{first of } E$
~~first (E) = T~~ $E \rightarrow T \Rightarrow E = \text{first of } T$

② Follow set → ignore any Rule
need first set upto to down

Follow (E) = { +,), \$ }
Follow (T) = { +, \$, * }
Follow (F) = { +,), \$, * }

direct copy coz nobody following

Follow (E) → right side e E k k follow korche like EU

e er exact right side e gach ache.

if terminal, directly follow set e bashabo or in Rule 1 → + E k follow korche Rule 3 →)

= follow set e kokhon o E thakbena.

= follow set e root not er fa follow set e always \$ hobe.

by default - \$ for root only

production body te E, T ache ki na. \$ root charao hoy but not default.

follow (T) → Rule 1, 4, 2 nobody is following T in 1, 2, 4 ∴ direct production head e jeye oitar follow set copy korbo in Rule 2, * follows T ∴ bashabo.

if non-terminal is following, tokon non-terminal er first set e ja ache ta copy korbo.

if first set e ϵ ache

$$\text{First}(F) = \{ (, id, \epsilon \}$$

$$\text{Follow}(F) = \{ (, id \}$$

$$T \rightarrow T * F ($$

$$T \rightarrow T * P id$$

$$T \rightarrow T * P \epsilon$$

means
hobday
following
F is
production
head er
follow set

copy korbo

eg) $\text{First}(A) = \{ num \}$

Follow

$$T \rightarrow T * B F E G$$

$$T * F (G$$

$$T * P id G$$

$$\text{Follow}(F) = \{ (, id, _ \}$$

$\downarrow$
 gger first set)
 or first set

SLR parse table

State id + * () \$ E E Goto state see automation for this.

State	id	+	*	(	)	\$	E	E	T	F
0	S5			S4			1	2	3	
1	S6	S6				Accept				
2		R2	S7		R2	R2				
3		R4	R4		R4	R4				
4	S5			S4			8	2	3	
5		R6	R6		R6	R6				
6	S5			S4				9	3	
7	S5			S4						10
8		S6			S11					
9		R1	S7		R1	R1				
10		R3	R3		R3	R3				
11		R5	R5		R5	R5				

follows of

follows of T to R1, R2, R3, R4, R5, R6

like LR parse table only diff in reduce. Shobal k R2/Rn likhbo. na. only production head e follow j ache tab set e R2 likhbo.

$E' \rightarrow E^0$
 $E \rightarrow E^1 + T^2 \mid T^3$
 $T \rightarrow T^3 * F^4 \mid F^5$
 $F \rightarrow (E) \mid id$
 $S \quad C$

- 1) $E \rightarrow E + T$ 5) $F \rightarrow (E)$
 2) $E \rightarrow T$ 6) $F \rightarrow id$
 3) $T \rightarrow T * F$
 4) $T \rightarrow F$

build this using LR parse table.

16/11/2024

id x id → LR diye possible na coz conflict.

stack	Symbol	Input Buffer	Action
0	\$	id * id \$	shift to 5
05	\$ id	* id \$	Reduce by 6 $F \rightarrow id$
03	\$ F	* id \$	Reduce by 4 $T \rightarrow F$
02	\$ T	* id \$	shift to 7
027	\$ T *	id \$	shift to 5
0275	\$ T * id	\$	Reduce by 6 $F \rightarrow id$
02710	\$ T * F	\$	Reduce by 3 $T \rightarrow T * F$
02	\$ T	\$	Reduce by 2 $E \rightarrow T$
02 01	\$ \$ E	\$	Accept

DFA generation in lexical analysis.
 Initially IB te id * id \$
 symbol → \$
 initially stack e 0
 shift korle stack e push.
 IB e kake dekhil stack e kake dekhil

1) 0 te id : # \$ sc
 2) 5 e *
 ∴ R

Reduce in means pop from stack.

05 → shift in prev
 . 4 push 5 in stack.

Stack theke 3 ta pop hobe
 coz T * 2 F
 reduce hache

table e Action ha thakle error.

parsing done.

Reduce hole jakete reduce korchi
 for jonno to stack the kono
 kichu nai use go to state.

check if stack empty or not

Reduce hole pop from stack

0 jonno go to 5 for jonno is 3

Stack top.

already go to 5

7 theke F dekhile to

Method → under a class
 Functions → not under class.

Local variable / global variable depends on scope

```
int c
{ int a b(---) }
```

```
int a;
}
c++;
```

→ ei scope er baire a nai.
 ce is a global scope here
 coz bracket er baire:

current function e kono new var jodi na pai, tokhon global e khuji shet function jodi arekta jetta amra use kori new variable palle direct global scope e khuji

Static scoping & dynamic scoping

↓
 1 ta ekta kore variables khuje
 like parent function e khujbe then grand parent function e if last function e ekta soddily variable pay.

Identifier distinguish karte lage.

Symbol → information about identifier
 coz bihbe na kon ID.

Symbol table implementation

↓
 implemented by
 1) List of Hashtables. 2) Hashtables of list.

IF of function.
 ta ekta while loop er bitor ache / arekta function e ache, tokhon ei regular baire jodi kono global scope thake oikhane jabe

float →

Symbol table

↓
stores info
of identifier
coz shobar
jonno ID
hoy amra
~~tabi~~ buta
bujhbo na
aet token
delehe
actual lexor
k chikho

Symbol table
scope tables.

Sum of we have a hashFunction like a linked list
eg → sum of ASCII values % scope-table-size.

Branch scope table as

Suppose $\text{my} = 0$ (with 5's)

redundantly
a label which is redundant) 0 index

in
2
20

ja ja
ID chib
stoblikhe
ashbo.

- Same index (scope) collision may or may not

int array insert

0
1
-16

19999, indexes

iterate = 10
if fail
abar
khuj
shub
large
iterate
work khuj
large
redundant
use
table

Worst case is last index
khali thakhe
∴ 1999 index
portanto jawab
1999.

Forward chaining suppose 1st index e a er info already ache ashakhon bashbo. last e jodi b insert korsi, jodi beshi piche hoy, shob go through kore jawa lagbe in the linked list. $\therefore$ so we keep new ID in head of index.

if for b, b k head banay add korbo (linked list) (1st address)
new keo ashle head e add korbo.
shamne theke add korbo.

Entry, exit, searching, & inserting $\Rightarrow$ default must.

different for all scope table.

Scope entry means I am creating a scope table/hash table.

scope entry,

0
1
2
3

If I say length = 4 $\rightarrow$ ami determine korsi
it means. $\&$ ja ja symbol table create hobe shobar jonno length = 4.

same scope & same variable name make error.

parser niger ekta variable rakhibe jar nam level = 0 $\rightarrow$ not a portion of my code.

same scope & same variable problem.

Scope entry $\Rightarrow$ create a scope table
& level er value 1 ta core increment korchir.

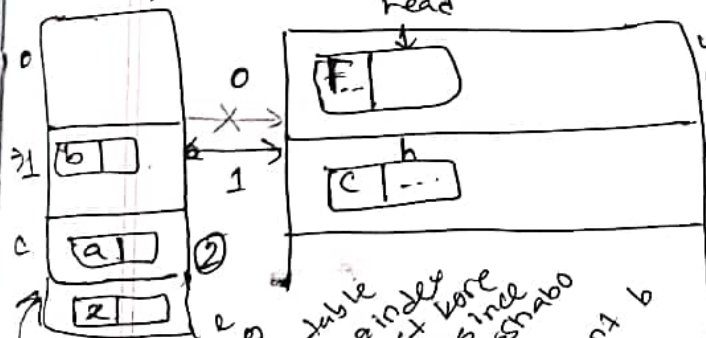
keyword

int c;
void f(int a, int b) {
 so read level = 1 c = 10

ekhan code thakbena token inakbe, f & k hash function e bashbo, suppose f er jonno index 1500.

scope table, hash-table same

same scope & same var er 2 ta name thakben



first e khorbo a, c ei table ei index e exist kore kina. since not tai bashbo

int a & int b er scope e exist kore. int c global scope e ache.

iccha motion a mar. eg. info, function, return, type, param, type, line

(same scope e same var name rakhte parba, so first check ei scope e same var exist kore kina. o te jeve linked list e check korbo f exist or not. & since nai, f & its corresponding attribute values store korbo)

last 5 min

Symbol table

- ↳ identifier er info store kori
- ↳ compiler er every phase er shathe symbol table connected
- ∴ every phase ei symbol table theke data extract korte pare.

2 types of implementations of symbol table

- ① List of Hashtables
- ② Hashtables of List.

code used

```
void f(int a, int b){  
    double x;  
    while (....){  
        int x, y;  
    }  
}
```

① List of Hashtables for this code

4 operations (for both list of hashtable & hashtables of list).

scope entry

process a declaration

process a use

scope exit

~~when~~

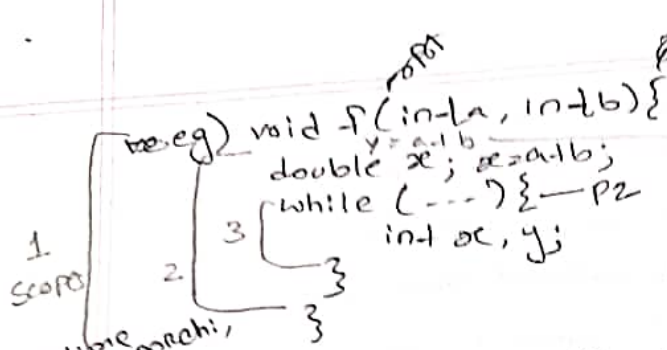
List of hashtables

↳ jolzhon new scope e enter kori, level = level + 1
∴ we create a new scope table.

↳ level → track rakhi kon scope e achi

scope table er length initialize kori

level=0



→ y to declared
na,
so baine
khuj bo
so arrow parent
scope en dike hobe.
table

first time
for code porchi,
o'ina
kono ekik
scope e
ache.

void er jonno new scope entry
"new scope table, level=1, length of
scope table is 4.

0	
1	
2	
3	

now insert value
in scope table.

for jonno token
is ID. (ki ki
attribute nibo bole
dibe).

now new scope for
P1; l=2, new table

process a declaration
khan kokhon scope
table e data insert
korte? after
first time declaration
• look up the
identifier in
the current scope
table

name er
variable,
tokhon
"multiple
declared
variable"
else
adding the
identifier
information
to the scope
table

when
seth is
initialized
for the
first time

void
(int a)
|
not
same b

if it is there,
void f(int a, int b, int b)
Same scope e same

erkhane jeto.
int a, b;
void f(int a, b)

if int x, y
a = 4

here a is not initialized
(a value is assigned).
so just check in the code if a exists in the code.

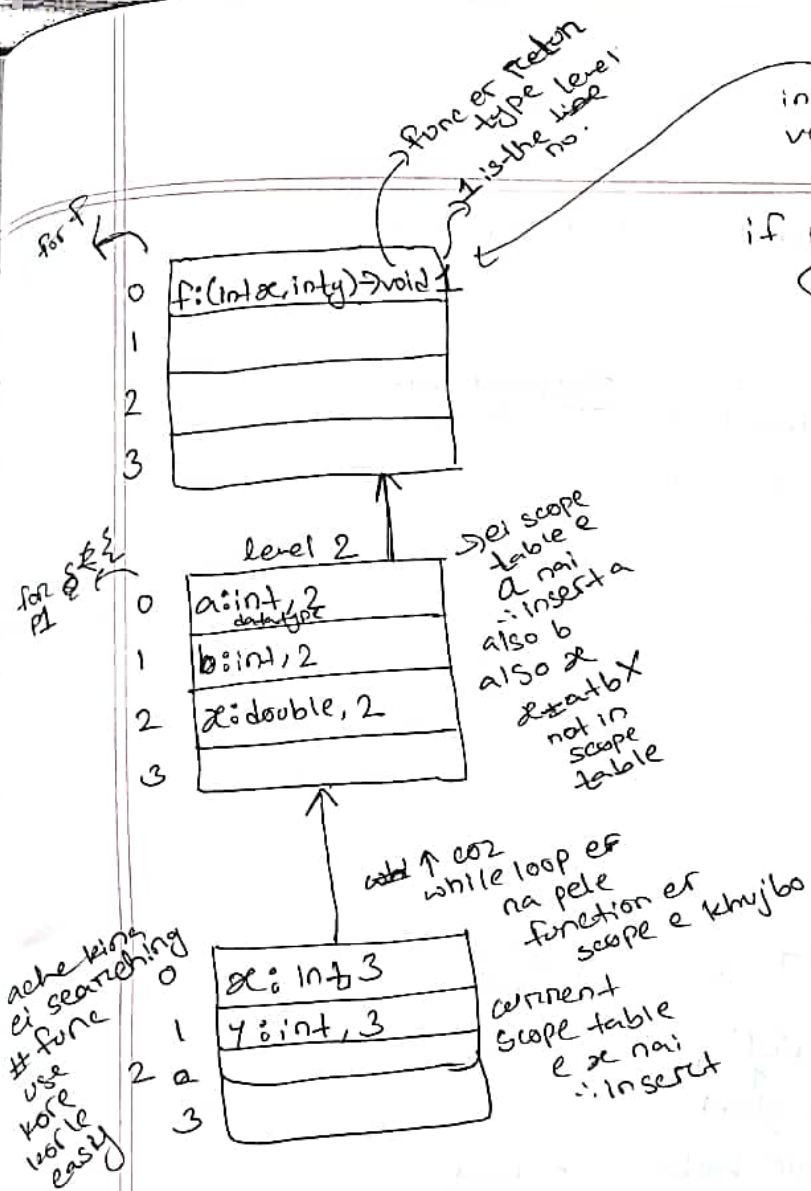
Process a use

suppose a = 4 a variable that actually exists in code a in function here.

void fb(---) {
void f(int a, int b)
if a erkhane initialize koto amra a er kono access e petam na.

process a use

look up the identifier in the current scope table, if it is there, then "fine" if it is not there, go to the parent scope, parent er kache na thakle go to grand parents finally na pele "Undeclared variable"



scope exit

level --
delete current scope table

} level = 2

} level = 1

global scope done
level = 0

function er scope.

ache king ei searching use kore wole easy

Hashtables of list

1. In hashtable, list of list aakar e data rakhke.

Similar these 4 steps.

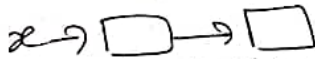
- (i) scope entry
- (ii) process a declaration
- (iii) process a use
- (iv) scope exit.

code

```
void f(int a, int b){
    double x;
    while (1){
        int x, y;
        a = 4;
    }
}
```

(i) scope entry $\rightarrow$ level ++ $\uparrow$ f
 level = 0 $\uparrow$ 1 $\downarrow$ 2 $\downarrow$ 3
 For global $\uparrow$ for loop scope.

Identifier for f, scope table e store korbo as a key
 value pair -
 attribute values gula er bitor link list aakar
 e data rakhbo.



kon level e ache.
 ota linked list hishabe rakhbo.

check if f is already in the scope table as key.
 a name or no key. $\therefore$ a insert.

multiple
 declared
 variable
 not only
 for same
 key.

* check the
 level
 linked
 list er
 kono node
 e level match
 kore kin

must level
info-traka
img loc.

same
level
e same 2 var
thm ka for
-bana

hash table len = 8

index
↓
0

0	a: ^{level} [int, int, void, 1]	
1	b: [int, 2]	[double, 2]
2	b: [int, 2]	
3	x: [double, 2]	head [double, 2] int, 3
4	y: [int, 3]	head current after 3, 3
5		
6		
7		

level 2
diye
a er jonno
kono
ache
kinn
check

check
if b
is key
already

int 3 → double 2

x already a key, check x er
attribute 3 ache kina 2 as level
→ if no, → add a node.

} → closing

} scope ext: level --

level = 9, 8, 3, 2, 1

32 it means 3rd scope er jonno julo node dhilo
destroy.

process a use
a 24
3 theke 2nd level
e khujbo
if not found
1st level e
khujbo

Semantic Analysis Phase

5/12/2024

SDD
syntax directed
definition

↓
CFG + semantic
rules.

$SDD \Rightarrow CFG + \text{semantic rules}$

SDT
syntax directed
translation

→ input parse tree
output " "

* SDD f upor base kore jei translation
process kori that is SDT.
SDD use kore jei attribute evaluate
kori that is SDT.

SDD $\Rightarrow$ Syntax Directed Definitions are a generalization
of context-free-grammars in which:

1. Grammar symbols have an associated set of attributes
2. Productions are associated with semantic rules for computing the values of attributes.

→ grammar symbols
non-terminal or some terminals.

→ Attribute gular shathe compute korar jonno.

~~Remember~~
SDD

Syntax analysis phase e we had cfg, along with those cfg,
we have semantic rules.

output is an annotated parse tree.

→ * Amar j attribute value gula thakbe tader k compute korar
jonno prottekta production rule er corresponding semantic
rules thakbe. (not necessary j shobar e thakbe).

SDD

CFG₂

$$S \rightarrow E \quad (1)$$
$$E \Rightarrow E1 + T \quad (2)$$
$$E \rightarrow T \cdot \textcircled{3}$$
$$T \rightarrow T \mid * F \quad (4)$$

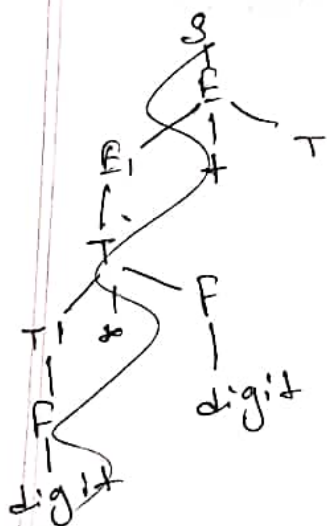
$T \rightarrow F$ (5)

$F \Rightarrow \text{digit } 6$

Parse Tree



अध्याय 2



eg) $E \rightarrow E + T$

semantic role

$$E.val \rightarrow E.val + T.val$$
$$E.val \rightarrow E1.val$$
 $+T.val$

sample
inhibitor

$$\overline{E}_{val} = E_{val}$$
$$E_{\text{val}} = E_{\perp, \text{val}}$$

↓
slider

valve

parent
e. l. schabk. o

ei $\leq$ final er value

i.e 4 etha am

operer £-val e boshabo.

+ $\frac{1}{T}$ $\rightarrow$ parent could differentiate

kecuali itu. Ed. 11 km

As for (see below needed)

no E2 (no +10511-2)

2. 2 E1 parent & hole child E
" " " E1

2 f "

11 11 E.1

parent
e val
parent e boshba
naki child er
val parent e
boshba
oit $\rightarrow$ differentiate
korib jonmo

Parse tree

SDD or CF + semantic rules

given

Semantic rule SDD

$S.val = E.val$ (Print(1))

$E.val = E1.val + T.val$

$E.val = T.val$

$T.val = 20$

$T.val = T1.val * F.val$

$T1.val = F.val$

$F.val = digit.lexval$

CFG

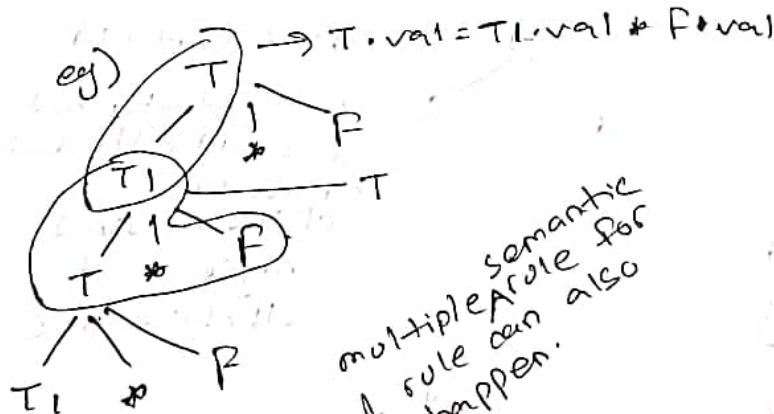
$E \rightarrow T$

$\{E.val = E1.val + T.val\}$

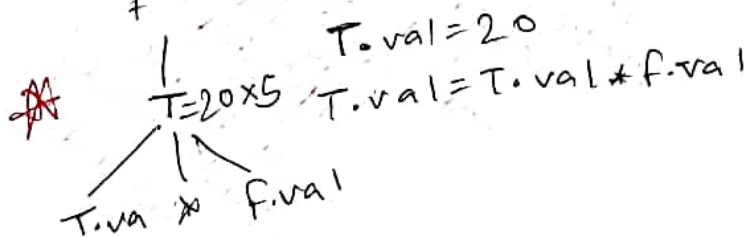
$\downarrow T$

$F \rightarrow digit \rightarrow$ semantic rule
is $F.val = digit.lexval$

$T \rightarrow F.val$ $T1.val = F.val$



semantic rule for multiple rule can also happen.



Semantic action

$\star$ print(1)

It means $\star$ jolkhon
s of reduce hoy, ami tokhon
tar attribute val cale
korte shi $\star$ tar sh
ekishathe $\star$ print korte shi

Annotated parse tree SDD

For attribute is val here.

SDD using Korkun

means tree banash time e $\star$ detach the perform the action then $\star$ korchhi in tree
digit.lexval means digit er actual value
 $\star$ nite chahi



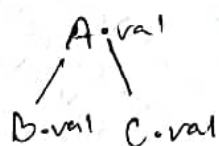
Attributes.

Two types of Attributes

1) Synthesized Attribute: A synthesized attribute at Node N is defined only in terms of attributes values at the children of N & N itself.

2) Inherited Attribute, An inherited attribute at Node N is defined only in terms of attribute values at N 's parent, N itself and N 's siblings.

$A \Rightarrow B, C$



≠

inherited

child, parent, parent, sibling and itself

if consider B ,

depends on $A, B, \& C$.
 $\downarrow$ parent $\downarrow$ sibling

synthesized

$A \Rightarrow B, C$



child & itself

A is N attribute & B, C are N attribute or value child of and nija kach theke ashbe

if A aro onno jaygay thake, shob gula khetre shob jaygay kore upar depend kore tar upar depend kore we'll decide synthesized / inherited.



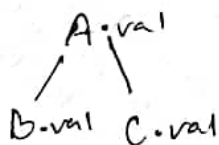
Attributes.

Two types of Attributes

1) Synthesized Attribute: A synthesized attribute at Node N is defined only in terms of attribute values at the children of N and N itself.

2) Inherited Attribute, An inherited attribute at Node N is defined only in terms of attribute values at N 's parent, N itself and N 's siblings.

$A \Rightarrow BC$



≠

inherited

child, parent,
parent, sibling
and itself

if consider
B,

depends
on $B \Rightarrow N$

$A, B, \& C.$
↓
parent sibling

synthesized

$A \Rightarrow BC$



child & itself

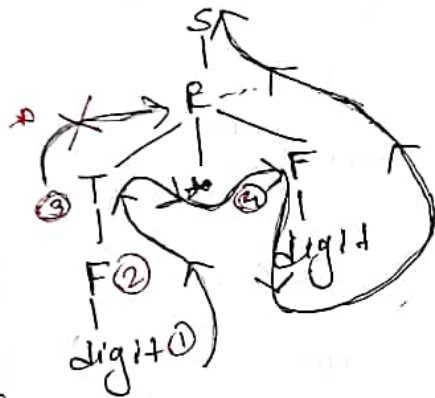
A is N attribute
& B, C are N attribute
or value
child of and
niger kach
theke ashbe.

if A also
onno jaygay
thake, shob gulab
khetre
shob gulab
test jaygay
more depend
more far
more depend
well decide
synthesized /
inherited.

Dependency Graph

↳ gives info of order of dependency graph.

→ A dependency graph helps us to determine how those attribute values can be computed. It tells us about which order we should evaluate the tree.



direction
gula dena
compiler
ke bole dena
laghe.

Koi theke koi jabo oi order
ta bole.

leaf node theke koi
jabo bla bla.

no. diye na dileo hole
arrow diye dilbo.

① → ② → ③

arrow
depends
on attribute
has no attribute

the arrow dena na dena
same

T & F purata ekshathe dekhbo.
i.e. Fami khali T er jonno dekhbo na

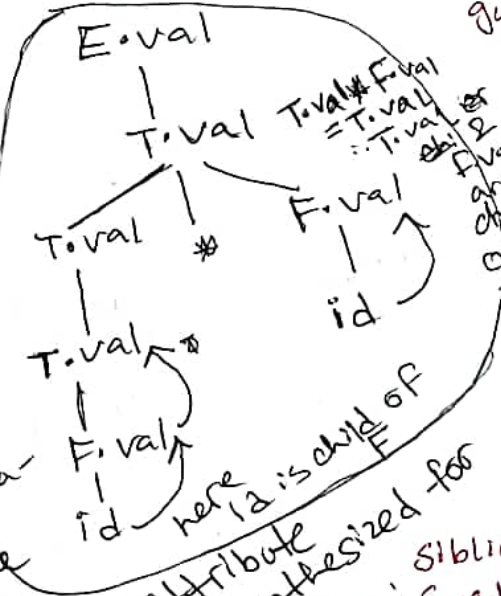
S attributed definition - An SDD is S attributed if every attribute is synthesized. S attributed SDD's are evaluated in bottom up parsing as values of the present node depend upon the values of the child node.

Q → Dependency
for a given
input string

2 CFG
Draw parse
tree & then
give arrows for
dependency

graph dekhbo
kon order e evaluate
bolle evaluate kora
lagbe na

detect
S attributed
def kima
every
node er
ja ja
attribute
value
ache
olgu
koi
theke
stem va-
ted
rocche
show



non-terminal
er attribute
gular value
cale korar
jonno
kotha
theke access
kora lagse
in the
last class
CFG

only access
from child

not parent

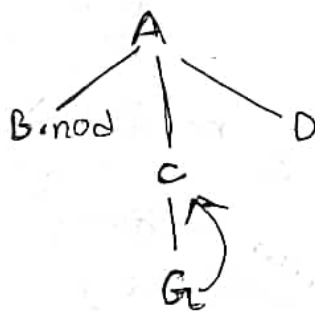
∴ synthesized

val attribute
is synthesized
all here

bottom-up
is basically
S-attributed

L-attribute SDD Definition: If an SDD uses synthesized attributes and inherited attributes with a restriction that inherited attribute can inherit values from left sibling only, it is called an L-attribute SDD.

Q. jodhon
C te reduce
hocche
B. node er
value er node
C er attribute
node e
red boshbe



$C \Rightarrow G$ C. node = B. node

L-attribute
e khetre
attributes
can be both
synthesized &
inherited

↓
inherited hote
hole only left
sibling theke
access korte
parbe.

jodi
emon
kichu
page dekhi
na, tokhon
ota root
node e
refer
kore.

Practice Prob

$$S \rightarrow E n$$

$$E \rightarrow E1 + T$$

$$E \rightarrow T$$

$$T \rightarrow T1 * F$$

$$T \rightarrow F$$

$$F \rightarrow \text{digit}$$

$$F.\text{val} = \text{digit}.\text{lexval}$$

eg string:

$$\text{digit} * \text{digit} + \text{digit} n$$

$$3 * 5 + 4n$$

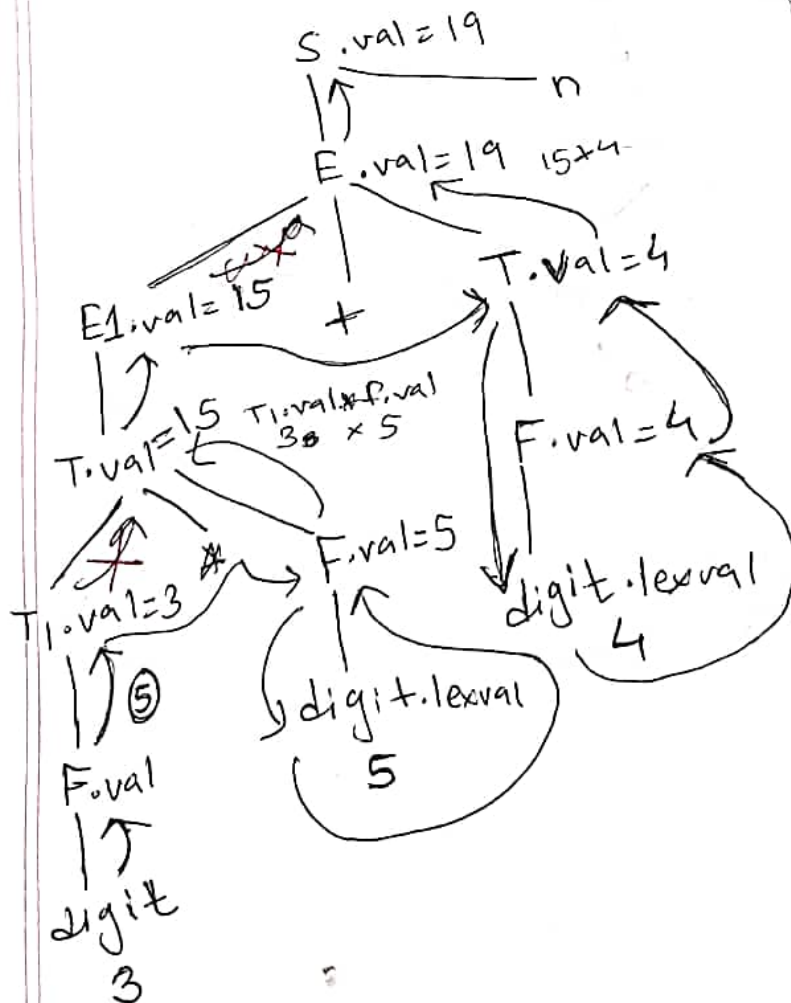
$$S.\text{val} = E.\text{val} \quad ① \quad 1$$

$$E.\text{val} = E1.\text{val} + T.\text{val} \quad ② \quad 2$$

$$E.\text{val} = T.\text{val} \quad 3$$

$$T.\text{val} = T1.\text{val} * F.\text{val} \quad 4$$

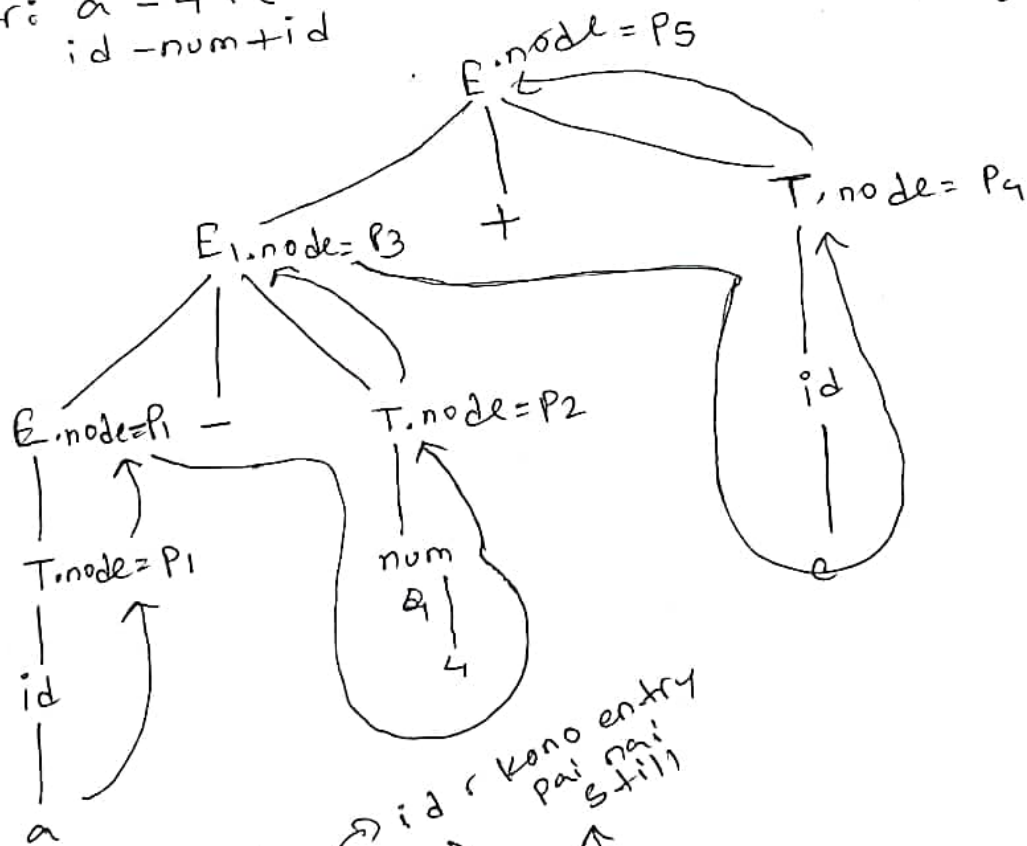
$$T.\text{val} = F.\text{val}$$



$E \rightarrow E_1 + T \rightarrow E.\text{node} = \text{newNode}(' + ', E_1.\text{node}, T.\text{node})$
 $E \rightarrow E_1 - T \rightarrow E.\text{node} = \text{newNode}(' - ', E_1.\text{node}, T.\text{node})$
 $E \rightarrow T \rightarrow E.\text{node} = T.\text{node}$
 $T \rightarrow (E) \rightarrow T.\text{node} = E.\text{node}$
 $T \rightarrow \text{id} \rightarrow T.\text{node} = \text{newLeaf}(\text{id}, \text{id}.\text{entry})$
 $T \rightarrow \text{num} \rightarrow T.\text{node} = \text{newLeaf}(\text{num}, \text{num}.\text{val})$

ekta class object create korch
 ↓

str: a - 4 + c
 id - num + id



id kono entry
 pai nai
 still

$P_1 = (\text{id}, \text{enter } a)$
 $P_2 = (4, \text{num}, 4)$
 $P_3 = (' - ', P_1, P_2)$
 $P_4 = (\text{id}, \text{enter } c)$

$P_5 = (' + ', P_3, P_4)$

$E \rightarrow E_1 + T \rightarrow E.\text{node} = \text{newNode}(' + ', E_1.\text{node}, T.\text{node})$

$E \rightarrow E_1 - T \rightarrow E.\text{node} = \text{newNode}(' - ', E_1.\text{node}, T.\text{node})$

$E \rightarrow T \rightarrow E.\text{node} = T.\text{node}$

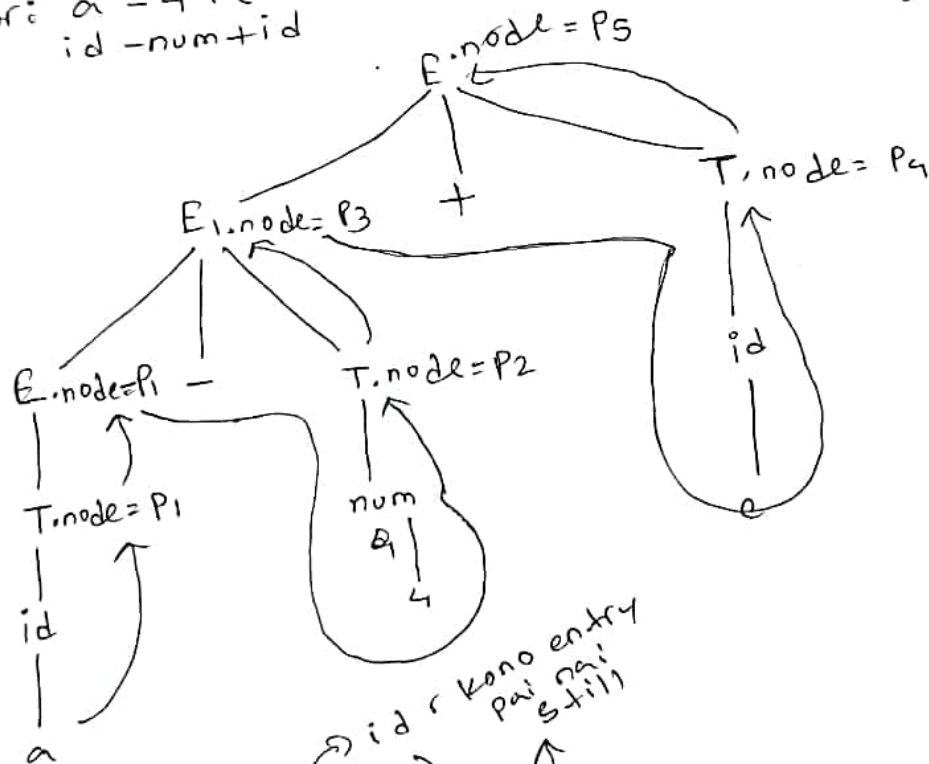
$T \rightarrow (E) \rightarrow T.\text{node} = E.\text{node}$

$T \rightarrow \text{id} \rightarrow T.\text{node} = \text{newleaf}(\text{id}, \text{id}.\text{entry})$

$T \rightarrow \text{num} \rightarrow T.\text{node} = \text{newleaf}(\text{num}, \text{num}.\text{val})$

str: a - 4 + c
id - num + id

ekta class object create korch



$P_1 = (\text{id}, \text{enter } a)$
 $P_2 = (4, \text{num}, 4)$
 $P_3 = (' - ', P_1, P_2)$
 $P_4 = (\text{id}, \text{enter } c)$

$P_5 = (' + ', P_3, P_4)$

id, kono entry pai nai still

num val jani here



B Type expression.

Basic type $\rightarrow$ int, float

composite type $\rightarrow$ array, linked list.

Q: expression
e which
ones are
basic
2 which
are composite
type

$T \rightarrow BC$

$B \rightarrow \text{int}$

$B \rightarrow \text{float}$

$C \rightarrow [\text{num}]C_1$

$C \rightarrow \epsilon$

$T.t = C.t$

$C.b = B.T$

$B.t = \text{integer}$

$B.t = \text{float}$

$C.t = \text{array}(\text{num.val}, C_1.t)$

$C.t = C.b$

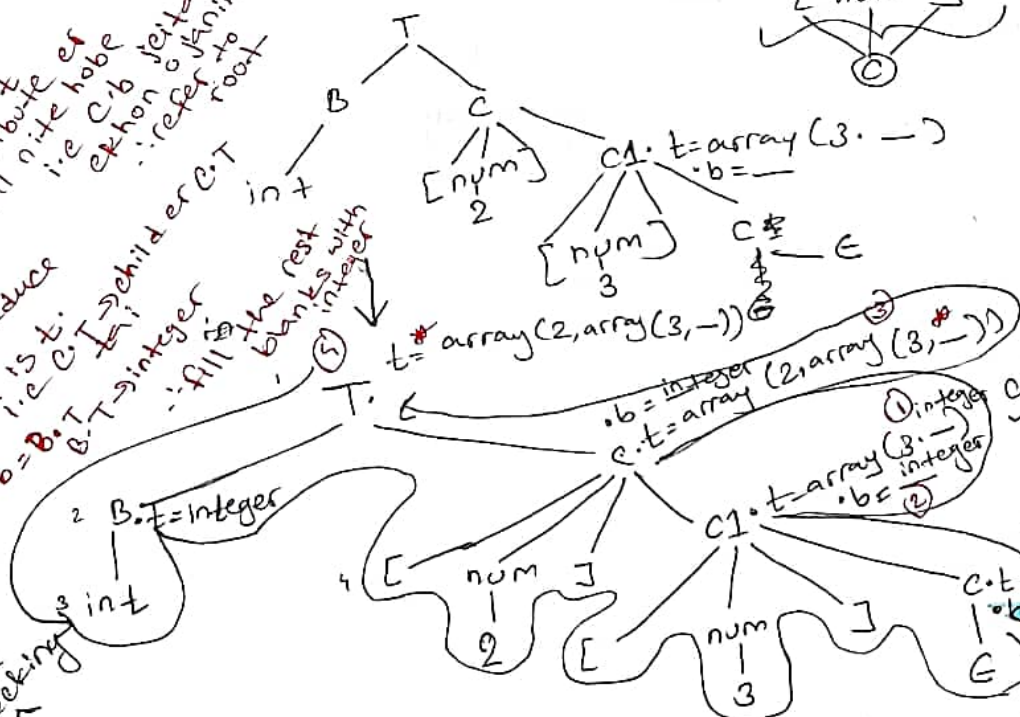
int [2] [3]

$\Rightarrow$ stream of tokens int [num] [num]

3# are separate.



1. $C_1.t$ is child of T .
attribute of T is $C_1.t$.
val nite hobe i.e. $C_1.b$ refer to root.
2. same
3. same
4. B & C are same.
T or C_1 to reduce.
the $C.b = B.T$.
i.e. $C.T$ is child of $C.T$.
B.T is integer.
fill in the rest with integer.



can extend
use it
for mem-
ory
allocation
& error
detection

type checking

type expression

$T.t = \text{array}(2, \text{array}(3, \text{integer}))$
 $\downarrow$
both composite & basic type

6th rule 2 apply
wise
 $C.b$ or $C.t = C.b$
i.e. $C.b$ val janning
node e refer
varbo.

11111111

Record structure

id, j, @ terminals

$$D \rightarrow T, d; D | E \in$$

$T \rightarrow BC \mid \text{record } \{ 'D' \}$

B 2:04 | -Noah

$$C \rightarrow C \mid [num]C \mid$$

```
input: int a;  
string: record
```

record

intc;

float b;

 $\{a, b\}$

→ here the record structure has 2 variables

record is a structure
name is ab
here ends } here.

~~ab~~
ab → name
id

if record
structure
or middle
array dthukay
dei.

int id;

record {

```
int id;
```

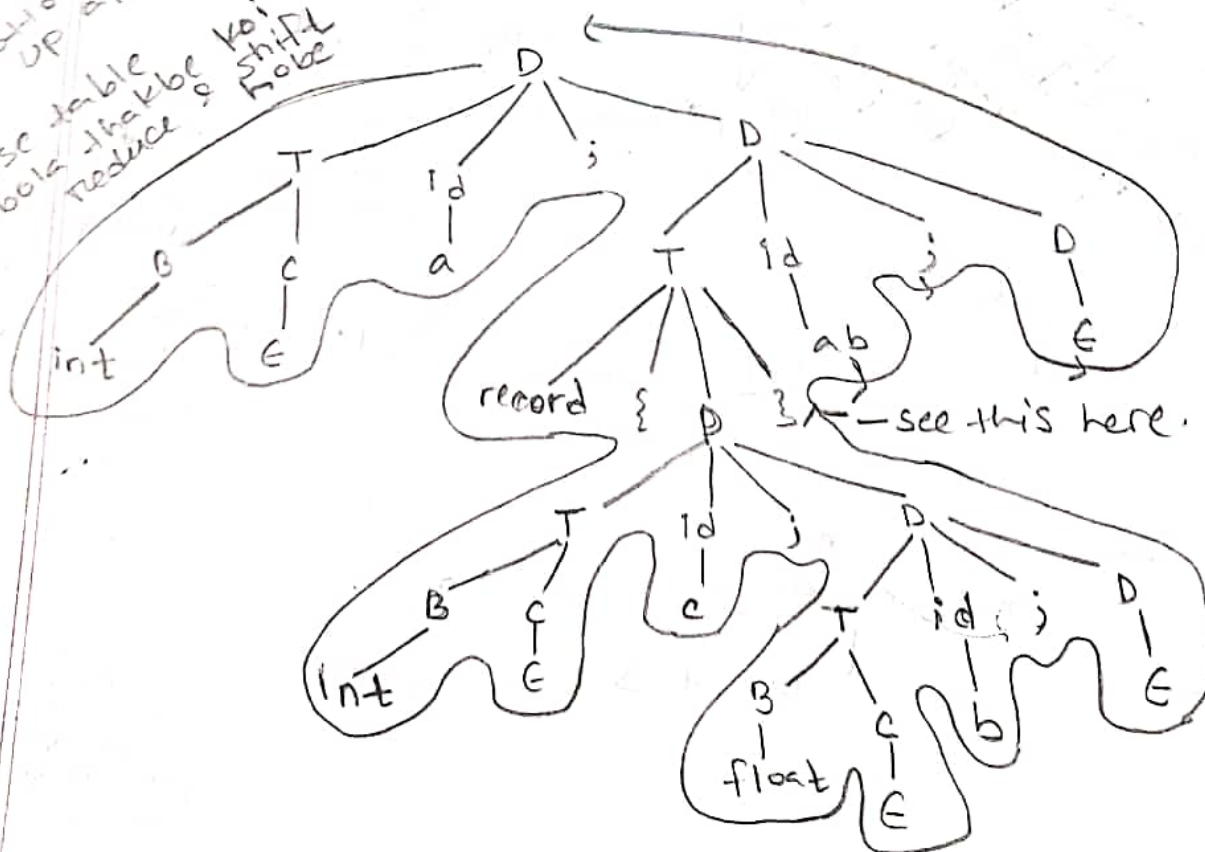
12

$$\{id;$$

→ Token
not given
do yourself.

Bottom up approach

Both are up at
 parse table
 e bolā thakbe
 reduce & ko shift
 move



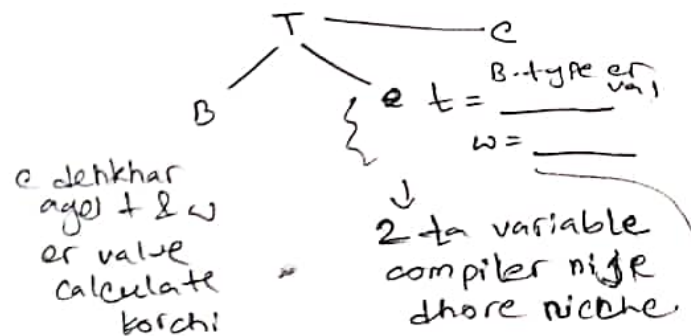
Practice

Input String:

```
int a;
record {
    int c;
    int [3][34];
} ab;
```

$T \rightarrow B \quad \{ t = B.type, w = B.width \}$
C

t & w are 2 variables.

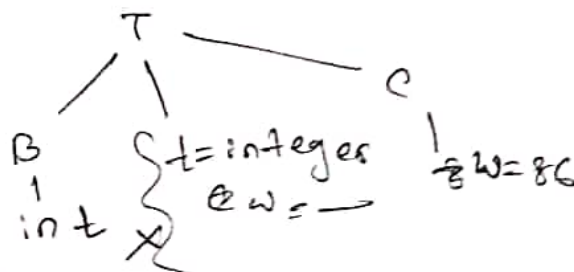


$T \rightarrow B \quad \{ t = B.type, w = B.width \}$
C

$B \rightarrow int \quad \{ B.type = integer \}$

$B \rightarrow float \quad \{ B.type = float \}$

t = integer
w = 2486



eikhaner t & w-er val change kore dite pare
... t & w bare kotha likhbo.

$T \rightarrow \{ \text{stack.push}(3) \} \quad B$

$C \rightarrow \{ \text{print}(2) \} / p1 = \text{newNode}(3, 4)$

$B \rightarrow \text{int} \{ B.type = \text{integer} \}$

$B \rightarrow \text{float} \{ B.type = \text{float} \}$

$C \rightarrow \{ B.w = 84 \}$

3



type expression $\rightarrow$ For type checking
memory allocation.

14/12/2024

* $\rightarrow$ 2 arrays & 3 columns $\rightarrow$ each column integer
array (2, array (3, integer)) $\rightarrow$ allocate memory for this type expression

for each integer
4 byte memory
allocated for each

$\begin{array}{|c|c|c|} \hline 4\text{ byte} & 4\text{ byte} & 4 \\ \hline \end{array} = 12\text{ byte}$

1) for every integer $\rightarrow$ 4 byte allocated.
" " float $\rightarrow$ 8 " "

what if array is $\text{int}[1000][4000]$?

$\Rightarrow$ for every each box 4 byte.

$\text{int}[2][3]$

$\begin{array}{|c|c|c|} \hline 4\text{ byte} & 4\text{ byte} & 4\text{ byte} \\ \hline 4\text{ byte} & 4\text{ byte} & 4\text{ byte} \\ \hline \end{array}$

total
24 byte for
this array

array (2, array (3, integer))

$\downarrow$

$\begin{array}{|c|c|c|} \hline 4\text{ byte} & 4\text{ byte} & 4\text{ byte} \\ \hline \end{array}$
12

$= 12\text{ byte for 1 row, } (3 \times 4) = 12\text{ byte}$

$\dots \rightarrow$ for 2 rows

$(12 \times 2) = 24\text{ byte.}$
byte

$3 \times 4\text{ byte} = 12\text{ byte.}$

$(12 \times 2)\text{ byte} = 24\text{ byte.}$

Sequence of Declaration

P $\rightarrow$ { offset = 0; } D

D $\rightarrow$ T

Stack $\rightarrow$ LIFO

Sequence of Declaration

~~$P \rightarrow \{ offset = 0; \}$~~

$P \rightarrow \{ offset = 0; \} D$

$D \rightarrow T id;$

$\{ top.put(id.lexeme, T.type, offset);$
 $offset = offset + T.width; \}$

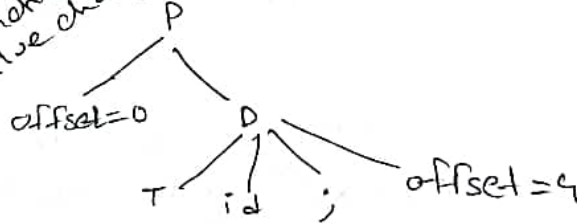
D_1

$D \rightarrow \epsilon$

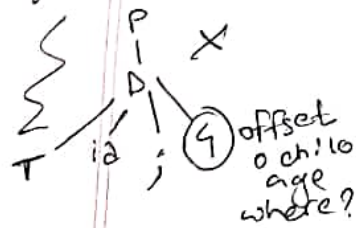
Production body to create variable and offset whose value is 0.

$\downarrow$
 not user given, compiler error var. stack

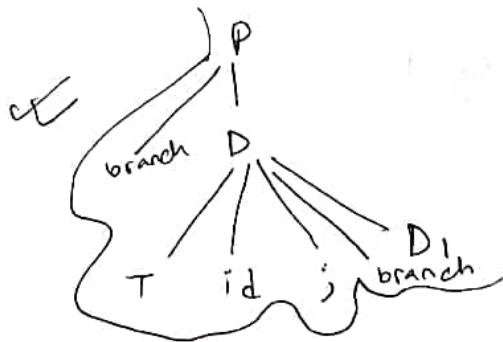
offset variable in same cache but value change
 cache is already a branch



first & middle create a branch.



Stack of stack mkn bo



$offset = 04$

~~D~~ $T \rightarrow D \mid T \text{ id } ; D \mid \epsilon$ array
 $T \rightarrow B \mid C \mid \text{record } \{ D \}$
 $B \rightarrow \text{int} \mid \text{float}$
 $C \rightarrow \epsilon \mid [\text{enum}] C \mid$

either
 don't
 sequence
 of declaration
 like no.

only non-terminal
 ex don't use for basic
 or composite
 type use record mention

Non-terminal D generates a sequence of declarations
 " " T " a basic, array and record type.
 " " B " basic type integer & float
 " " C " null string, composite type & array

(C) $\rightarrow$ both null and composite type...

$C \rightarrow \epsilon \mid \text{array}$

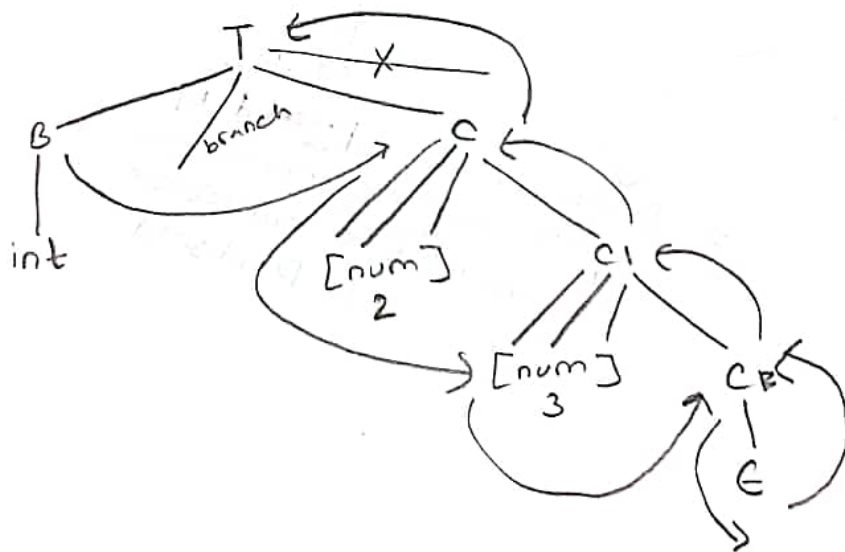
$\epsilon \rightarrow \text{null}$

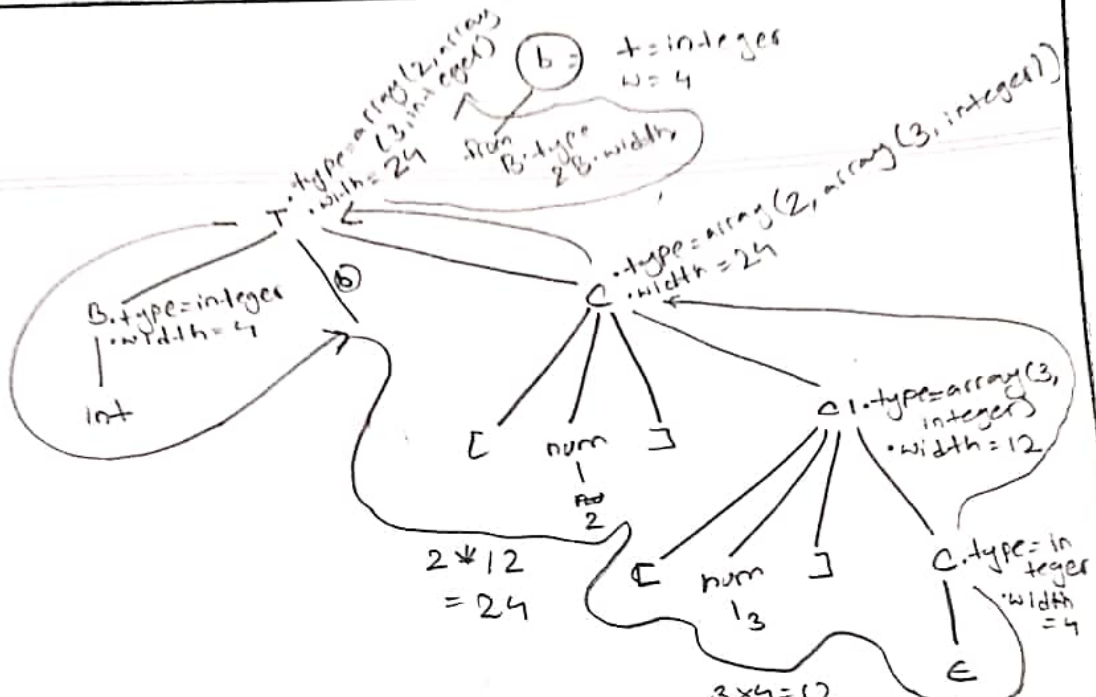
$\Rightarrow$ D generates $\text{record } D \mid$
 basic, array, record, identifier
 & D itself.

Syntax tree:
dedicated translation
for arrays

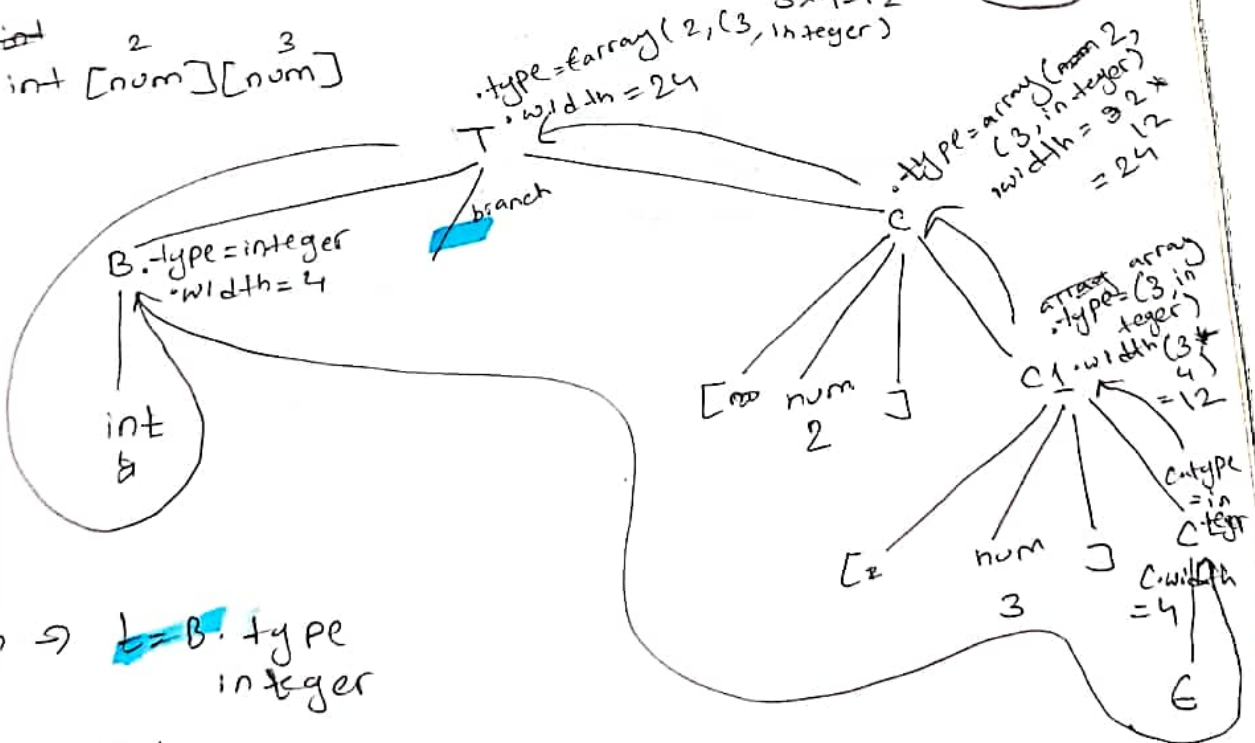
$T \rightarrow B \{ t = B.type, w = B.width; \}$ ^{branch}
 $C \{ T.type = C.type, T.width = C.width \}$
 $B \rightarrow int \{ B.type = integer, B.width = 4; \}$
 $B \rightarrow float \{ B.type = float; B.width = 8; \}$
 $C \rightarrow E \{ C.type = t, C.width = w; \}$ ^{?? kotho not mentioned}
 $C \rightarrow [num] C1 \{ C.type = array (num.value, C1.type); C.width = num.value * C1.width \}$

string: - int[2][3] int[num][num]
 translation





$\text{int } [num][num]$



branch $\rightarrow$ $b = B.\text{type}$
 integer
 $w = 4$

quiz
topic → array translation
& today
record included

Important
**

The Applications of types can be grouped under checking and translation.

1. Type Checking uses logical rules to reason about the behavior of a program at run time. Specifically, it ensures that the types of the operands match the type expected by an operator. For eg, the && operator in Java expects its two operands to be booleans and the result is also of type boolean.

22 TF
FF
TT

array(2, array(3, integer))
→ string

2. Translation Application: From the type of a name, a compiler can determine the storage that will be needed for that name at run time. Type information is also needed to calculate the address denoted by an array reference, to insert explicit type conversions, and to choose the right version of an arithmetic operators among other things.



24 byte
int[2][3]

Intermediate code generation $\rightarrow$ 4th stage.

21/12/24

Analysis $\rightarrow$ Frontend $\rightarrow$ Syntax $\rightarrow$ machine independent front end
 Synthesis $\rightarrow$ Backend $\rightarrow$ assembly code, machine code.

$\rightarrow$ shesh hole 3 address code pai

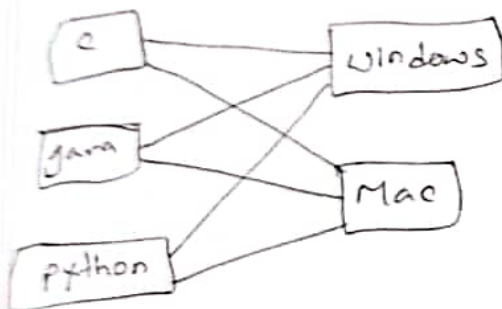
$\rightarrow$ " " machine code pai

Backend is machine dependent.

$\downarrow$
 kon machine e korchhi tar compiler
 er upor base kore.

means
 jei kons
 jaygar run
 kora jay
 eg $\rightarrow$ byte code.

$\rightarrow$ frontend.
 $\rightarrow$ Analysis
 portion



$\approx$ 3 languages er jonno 6 ta compilers
 lagche.

shobar jonno byte
 code generate
 korchun



ei byte code niye windows
 2 mac e run kor-bo

$\therefore$ 5 compilers needed now.

mxn compiler

$\rightarrow$ shobar jonno alda
 compiler laglo

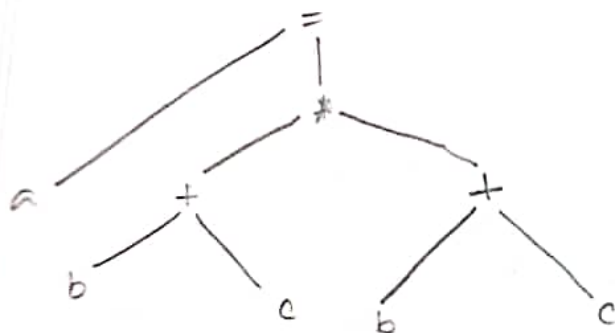
not now,
 byte code generate kore OS run kore
 $\therefore$ no. of compiler needed $\downarrow$.

Two representations
 Intermediate code generation can be represented
 by 3 address code & DAG (Directed Acyclic
 Graph)

↓
 maximum 3
 in code address
 = 3 or less

≅ 3 add code assembler etc for cache
 jabe

~~≅ eg~~ for the expr:
 eg) $a = (b+c) * (b+c)$ Draw DAG!



syntax tree → REG ex
 to DFA
 direct
 method

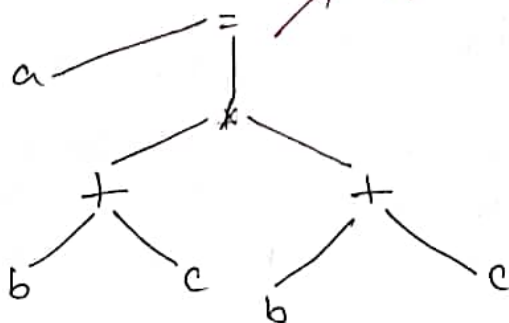
parse tree

has condition
 internal node
 e operator that be
 leaf node e letter
 no. blabla that be

parse tree → needs
 BCFG

internal node → operators
 (=, +, *)

leaf → a, b, c

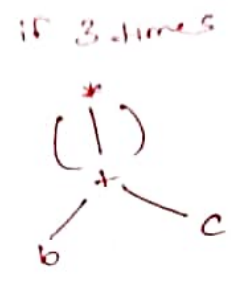
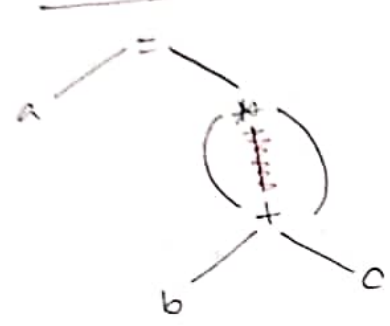


Syntax tree

↓
 internal node e
 operator that be
 and leaf node
 e letters
 and no. s
 bla bla

DAG is a sort of syntax tree
 but kono repetitive portion harkhena.
 ((here b & c is repeating))
 twice.

DAG



use array data structure for storing this DAG.

0	id i		
1	num 10		
2	+	index 0	index 1
3	i + =	0	2

* 1) first check data already in array/hashtable (ko repetitive khabena).
 since na hai we put i in 0 index (hashable thake hashfunc on usay index hoke).

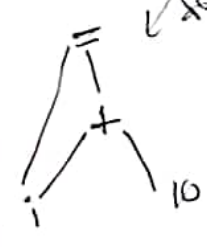
2) then $i = i + 10$
 age add korbo then assign korbo.

1 er info oth indexe
 2 to er info 1st " "

eg) $i = i + 10$
 syntax tree



DAG



for this see DAG

* same for num 10.
 3) now assign $i =$

array len here is 4, but array len 4000 hole
 i search korte time lagbe (lookup time ↑)
 ∴ use hashtable → more efficient.

practice

- ① Array representation for $a = (b+c) * d$
 $a = b+c * b+c$
DAG.
- ② Both DAG + Array representation
for $a = a + (b-c) + (b-c) * d$

DAG $\rightarrow$ disadv

line by line execute karna Lage
(no jump possible).

Small code $\rightarrow$ DAG better

Complex " $\rightarrow$ 3 add code better.

3 add code generation

eg) $z = a + b$ left to right jabo

$a + b$ add (add kore) z e assign korchhi.

$t_1 = a + b$ 3 add

t_1 (temp var jekhan add korchhi)

$z = t_1$ (2 add)

1 operator =

now z e assign

kortam (added sum ta).

eg) $z = (a * b) * c$

$t_1 = a * b$ 3 add

$t_2 = t_1 * c$ 3 add

$z = t_2$

conditions for

$t_1 = a * b$ 3 add

$z = t_1 * c$ 3 add

not optimized

optimized

$t_1 = a * b$
 $z = t_1 * c$

→ optimized one.

~~$t_1 = a * c$~~ coz left do right
 ~~$t_1 = a * b * c$~~ b age dkh-be
 $t_1 = a * b * c$ 3 add!

Conditions for 3 add code.

- 1) Highest 3 addresses can be there
- 2) At least 1 operator should be present.

use these conditions for writing 3 add code.

non-terminating,
or attribute
type / width
- make.

28/12/24

print(id.lexeme, value)

output
1, 0
print(1, 0) X

~~print(1, 0)~~

D → Tid {Symbol = SymbolTable.get(id.lexeme);

if id = x
∴ Symbol = x

Symbol = x

Symbol = SymbolTable.get(x)

~~new Env()~~

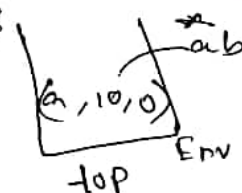
Assignment 3

T → record '{' {Env.push(top),
top = new Env(),
Stack.push(offset),
offset = 0}

D → top.push(id.lexeme,
T1.width, 10,
offset)

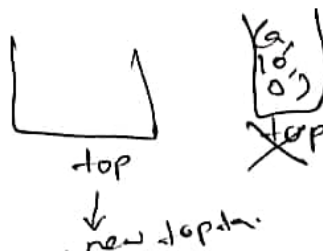
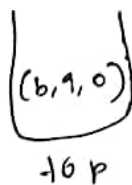
D → top.push(id.lexeme,
T1.width, 10,
offset)

offset = 10



{top = Env.pop(),
offset = Stack.pop(),
}

top = new Env() → env class
or obj
create
locally



2/1/2025

5

id id id lexeme

$a = b + c;$

Production

Semantic rule

means etta just normally boshbe.

$S \rightarrow id = E;$

$S.code = E.code \parallel \text{gen}(\text{top.get}(id.lexeme)) = E.addr).$

$E \rightarrow E_1 + E_2$

$E.addr = \text{new Temp}()$

$E.code = E_1.code \parallel E_2.code \parallel$

$\text{gen}(E.addr = 'E_1.addr$
generate function of $+$ $E_2.addr)$

$E.addr = \text{new Temp}()$

$E.code = E_1.code \parallel$

$\text{gen}(E.addr = 'minus' E_1.addr)$

$1 - E_1$

$1 - E_1$

$1(E_1)$

id

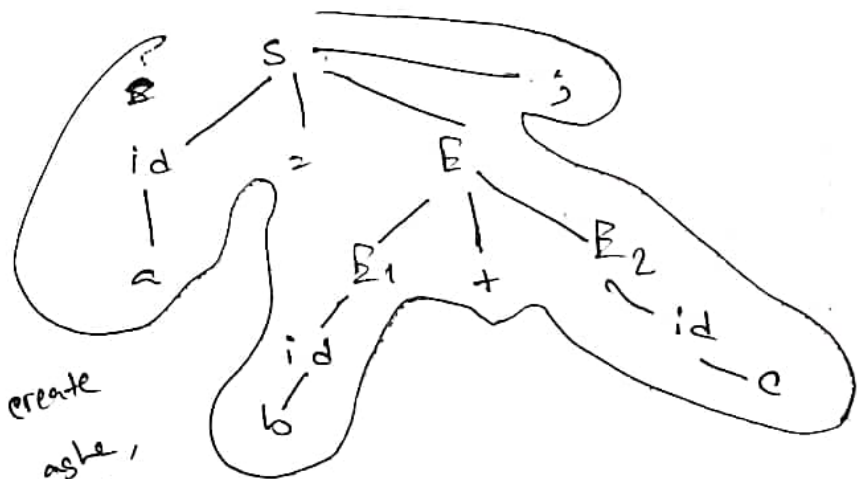
$E.addr = E_1.addr$
 $E.code = E_1.code$

$E.addr = \text{top.get}(id.lexeme)$
 $E.code = ''$

$\parallel$ concatenation sign

gen $\rightarrow$ function
for bhitorer
actions gula
generate kore.

$\text{new Temp}() \rightarrow$ temp
class er obj create
hocche
jokhon e etta aske,
it means temp
variable
generate hocche.



Relative address calculation for two dimensional array

row	column	0	1	2	3	4
row 0		1	2	3	4	5
row 1		6	7	8	9	10
row 2		11	12	13	14	15

every index
e integer
value is
which is
4 bytes.

60	15
56	14
52	13
48	12
44	11
40	10
36	9
32	8
28	7
24	6
20	5
16	4
12	3
8	2
4	1
0	

row major

40	9
36	4
32	13
28	8
24	3
20	12
16	7
12	2
8	11
4	6
0	1

column major

base = 0. → array ta jee jekhane start kore in memory.

2 dim array 2 types of representation possible

- ① row major
- ② column major

row major

row wise row es shob value rakho row 1 es shob val row 2 " " " so on

consider base { 4-7 e 2 pake 16-19 " 5 " 11

4 kore add kore memory location

row major important

Y[1][3]

1st row es 3rd column es value chai i.e. 9

→ eitei memory add e jeye value fetch kore.

col 4 byte kore jayga niche

Y[1][3]

for each row we need
4x5 = 20 bytes.

$$\text{base} + (\text{row size} * \text{row number})$$

$\downarrow$ $\downarrow$ $\downarrow$
 0 20 1

$$+ (\text{data size} * \text{column no.})$$

$\downarrow$
4

$$\Rightarrow \text{base} + (\text{row size} * \text{row no.}) + (\text{data size} * \text{column no.})$$

$$20 * \text{byte} * i_2$$

$$= 20 * 0$$

$$= 0$$

→ 1st index already 20 byte memory allocated here jese.

$$20 * 1 = 20$$

$$20 * 2 = 40$$

2nd row te jete jete 40 bytes at memory already allocated.

First index 1
e as has
20 byte
allocated

$$20 * 1 = 20 \text{ bytes}$$

first row er jonno already 20 bytes allocated.

2 * each index e there are 4 bytes integers.

$$4 * 3 = 12$$

$\downarrow$
column

$$20 + 12 = 32 \text{ bytes.}$$

$$\therefore \text{base} + 32$$

base theke 32 bytes dure

$$\text{base} = 0 \text{ jabo.}$$

$$32 + 0 = 32$$

array data structure
↓
direct index e
jete jete
val te
fetch korte
pare.

row major e
data kha jay
already
32 te 9
sabe.

Example 6.12: Let a denote a 2×3 array of integers, and let c , i , and j all denote integers. Then, the type of a is $\text{array}(2, \text{array}(3, \text{integer}))$. Its width w is 24, assuming that the width of an integer is 4. The type of $a[i]$ is $\text{array}(3, \text{integer})$, of width $w_1 = 12$. The type of $a[i][j]$ is integer .

An annotated parse tree for the expression $c + a[i][j]$ is shown in Fig. 6.23. The expression is translated into the sequence of three-address instructions in Fig. 6.24. As usual, we have used the name of each identifier to refer to its symbol-table entry. $\square$

```

S → id = E ;    { gen( top.get(id.lexeme) '=' E.addr); }

      | L = E ;    { gen( L.addr.base '[' L.addr ']' '=' E.addr); }

E → E1 + E2    { E.addr = new Temp();
                  gen( E.addr '=' E1.addr '+' E2.addr); }

      | id        { E.addr = top.get(id.lexeme); }

      | L          { E.addr = new Temp();
                  gen( E.addr '=' L.array.base '[' L.addr ']' ); }

L → id [ E ]     { L.array = top.get(id.lexeme);
                  L.type = L.array.type.elem;
                  L.addr = new Temp();
                  gen( L.addr '=' E.addr '*' L.type.width); }

      | L1 [ E ] { L.array = L1.array;
                  L.type = L1.type.elem;
                  t = new Temp();
                  L.addr = new Temp();
                  gen( t '=' E.addr '*' L.type.width);
                  gen( L.addr '=' L1.addr '+' t); }

```

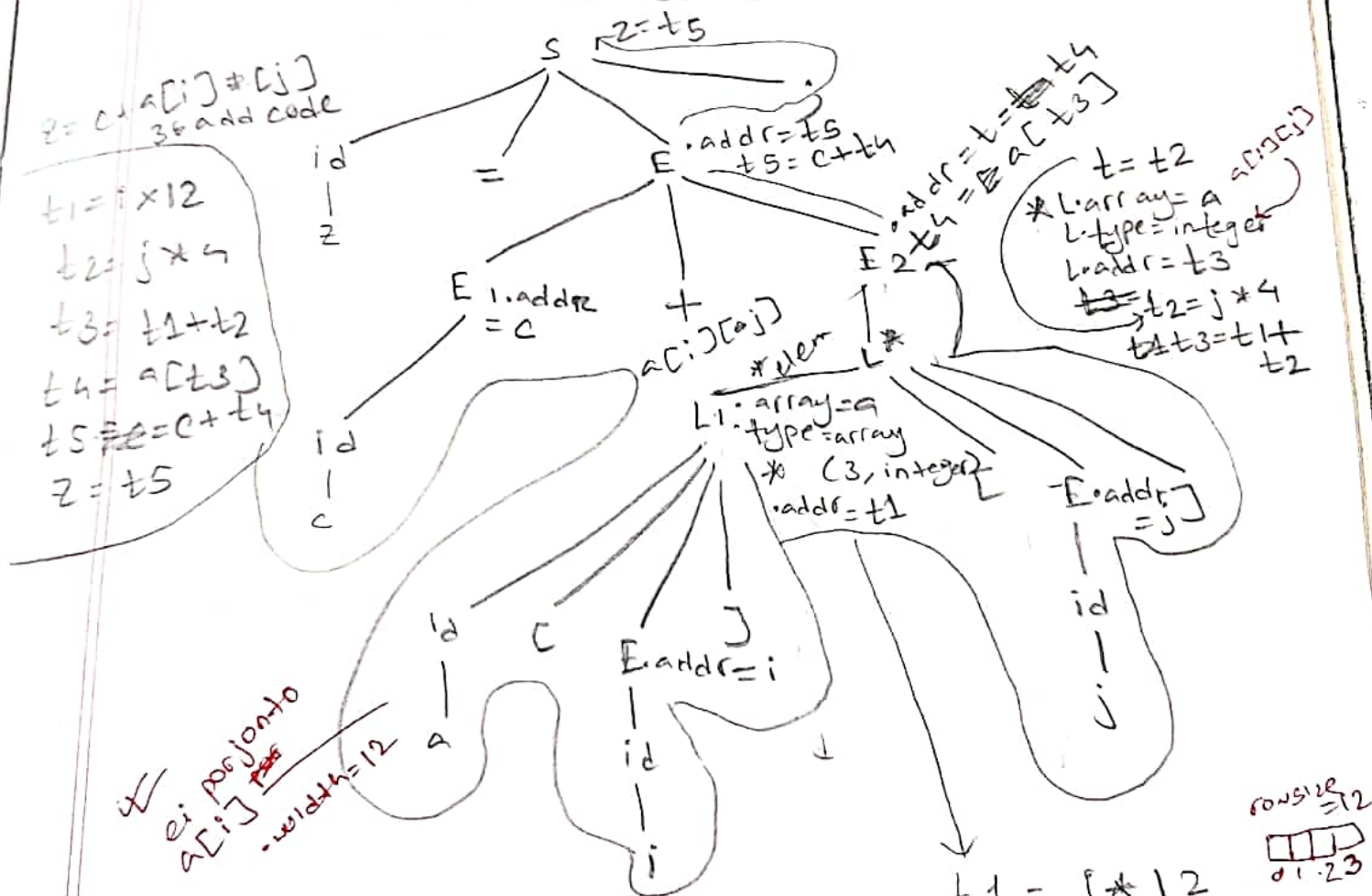
52

2 = 0 - 1 a [;] [;] ;

integer (integer) integer

A = array(2, array(3; integer))

base add
the name
of array

$$id = id + id[id][id]$$


* int [2][3]
array(2, array(3, integer))

a

~~a = 2~~

$t1 = 1 * 12 \rightarrow 3 \times 9$
 ↓
 col
 For each row
 1/2

3 array of integers
 of a is array(2, 07)
 h of an integer is 4.
 B1
 re type of a[1][3] is
 session e + a[1][3] i
 sequence of three and con
 te name of each ide - post

exrme) 'E' E.addr)

l.addr 'E' E

l.addr 'E' E

l.addr 'E' E

l.addr 'E' E

l.addr 'E' E

4/10/25 after code portion

* L1 = 200 (E1)

L2 = 100 (E2)

Equal code to portion

if L1 < L2 goto B1.true

goto B1.false

L3 = 200 (E3)

L4 = 200 (E4)

if L3 > L4 goto B3.true

goto B3.false

if L5 = Y goto B4.true

goto B4.false

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

code . B1

eg C.12:

Z = C + a [C] [C]

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

A = array(2, 07)

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

integer integer

eg 6.12:

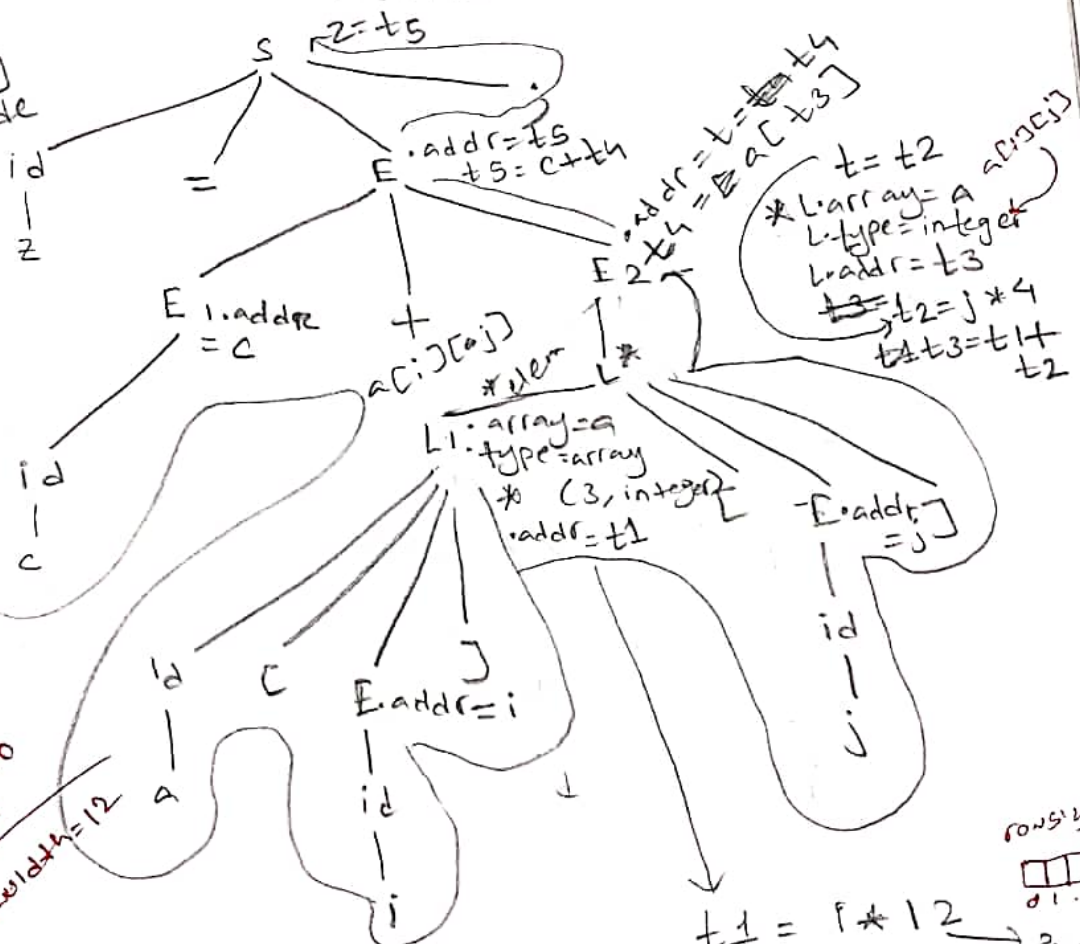
2x3 array
 $z = c + a[i][j];$
 Integer Integer Integer

$A = \text{array}(2, \text{array}(3, \text{integer}))$
 base address write the name of array

~~id = id +~~
 $id = id + id[id][id]$

$z = c + a[i][j]$
 3 add code

$t1 = i * 12$
 $t2 = j * 4$
 $t3 = t1 + t2$
 $t4 = a[t3]$
 $t5 = c + t4$
 $z = t5$



✓ ei points to $a[i][j]$
 width = 12

* $\text{int}[2][3]$
 $\text{array}(2, \text{array}(3, \text{integer}))$
 A

$t1 = i * 12$
 read info in question

consider 12
 0 1 2 3
 3x4
 ↓
 col

For each row

Control flow &

↳ for if, else, while, statement (or translation)

Boolean operator are applied to ~~boolean variables~~

① Boolean variables

② Relational expression with relational operator.

In programming languages, boolean expressions are often used to ~~alter the flow of control~~

① Alter the flow of control: Boolean expressions are used as conditional expressions in statements that alter the flow of control.

eg: IF (E) ^{execute} True hole statement, ^{hobe} S

E true hole S hobe

E false " S hobe na

if ($a > b$) { $x = 0$ }

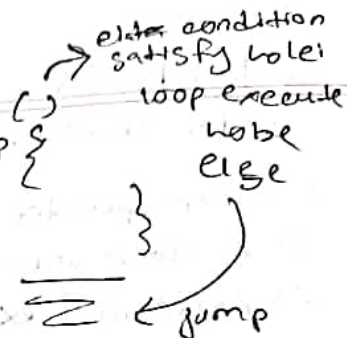
* jodi $a > b$ false $x = 0$ hobe na

② ~~Comp~~ Compute logical values: A boolean expression can represent true or false.

logical val compute kora
janno boolean expression
compute korte parbo.

Short circuit code / Jumping code.

while loop



&, ||, ! $\rightarrow$ jump & translate hoy

expression gula position

diye represent kora hoy

L2

eg) if (^a $x < 100$ ^b $||$ $x > 100$ ^c $&$ $x \neq 4$)
 $\underbrace{\hspace{1.5cm}}_T$ $\underbrace{\hspace{1.5cm}}_T$ $\underbrace{\hspace{1.5cm}}_T$ { $x = 0;$ }

True hole $x=0$

else $\underbrace{\hspace{1cm}}_{\text{normal lines e jabe}}$

or
 $T \vee T \rightarrow T$
 $T \vee F \rightarrow T$
 $F \vee T \rightarrow T$
 $F \vee F \rightarrow F$

a $\rightarrow$ true hole $x=0$ hole
 baki r dekha lagbena

$T \& T \rightarrow T$
 $T \& F \rightarrow F$

a $\rightarrow$ false hole b & c check korta jabe

b & c, T, T hole
 $b \& c = 0$ hole
 coz both b & c
 & hoye ase

$$(x+k < 100 \parallel x+k > 200 \&\& x \neq y)$$
$$01 + x = k$$
$$y = x + 10$$


PRODUCTION	SEMANTIC RULES
------------	----------------

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = \text{undefined}$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel \text{label}(B_1, false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = \text{undefined}$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel \text{label}(B_1, true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel \text{gen}'if' E_1 \text{ addn rel op } E_2 \text{ addn 'goto' } B.true$ $\parallel \text{gen}'goto' B.false$
$B \rightarrow true$	$B.code = \text{gen}'goto' B.true$
$B \rightarrow false$	$B.code = \text{gen}'goto' B.false$

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} * E :$	$S.\text{code} = E.\text{code} \parallel$ $\text{gen}(\text{top.get}(\text{id.lemma})') = E.\text{addr}$
$E \rightarrow E_1 + E_2$	$E.\text{addr} = \text{new Temp}()$ $E.\text{code} = E_1.\text{code} \parallel E_2.\text{code} \parallel$ $\text{gen}(E_1.\text{addr}' = E_1.\text{addr}' + E_2.\text{addr})$
$E \rightarrow E_1$	$E.\text{addr} = \text{new Temp}()$ $E.\text{code} = E_1.\text{code} \parallel$ $\text{gen}(E_1.\text{addr}' = \text{'minus' } E_1.\text{addr})$
$E \rightarrow (E_1)$	$E.\text{addr} = E_1.\text{addr}$ $E.\text{code} = E_1.\text{code}$
$E \rightarrow \text{id}$	$E.\text{addr} = \text{top.get}(\text{id.lemma})$ $E.\text{code} = \text{' '}$

2.

2720000

6950
79, 8
45, 1
02, 3
0

Stedman's
26 gram inch

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S_{\text{next}} = \text{newlabel}()$ $P.\text{code} = S.\text{code} \parallel \text{label}(S.\text{next})$
$S \rightarrow \text{assign}$	$S.\text{code} = \text{assign.code}$
$S \rightarrow \text{if } (B) S_1$	$B.\text{true} = \text{newlabel}()$ $B.\text{false} = S_1.\text{next} = S.\text{next}$ $S.\text{code} = B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$B.\text{true} = \text{newlabel}()$ $B.\text{false} = \text{newlabel}()$ $S_1.\text{next} = S_2.\text{next} = S.\text{next}$ $S.\text{code} = B.\text{code}$ $\parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$ $\parallel \text{gen}(\text{goto } S.\text{next})$ $\parallel \text{label}(B.\text{false}) \parallel S_2.\text{code}$
$S \rightarrow \text{while } (B) S_1$	$\text{begin} = \text{newlabel}()$ $B.\text{true} = \text{newlabel}()$ $B.\text{false} = S.\text{next}$ $S_1.\text{next} = \text{begin}$ $S.\text{code} = \text{label}(\text{begin}) \parallel B.\text{code}$ $\parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$ $\parallel \text{gen}(\text{goto } \text{begin})$
$S \rightarrow S_1 S_2$	$S_1.\text{next} = \text{newlabel}()$ $S_2.\text{next} = S.\text{next}$ $S.\text{code} = S_1.\text{code} \parallel \text{label}(S_1.\text{next}) \parallel S_2.\text{code}$

$$F =$$

91.

E. add.

$t = t_2$
 $\text{wall} = \text{int} \cdot t_2$
 $\text{wall} = t_3$
 $\text{wall} = t_2$
 $\text{wall} = t_3$

if ($x < 100$ || $x > 100$ & $x \neq y$) $x = 0$;

if

if true false lie jabe
if $x < 100$ goto L1
= if false $x > 100$ goto
= if false $x \neq y$

L1: $x = 0$

L2: =

jump to L2
cor F&F

since false
 $x = 0$
not be
- L2 te jabe.

F & T

F & F = F

Flow of control statement

for else if 1) $S \rightarrow \text{if } (B) S_1$ Statement
boolean exp

$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$

$S \rightarrow \text{while } (B) S_1$

3 grammars
for flow
of control.

B represents boolean
S represents statements

B er joinno attribute B.code
B.true/B.false

S er attribute code, S.code

they
containing
codes
of B & S
& they
manage code
manage
codes

jumping label labels are managed
using B.true $\rightarrow$ next
B.false
S.next

Jumping labels are managed using
 B = true $\rightarrow$ label to which control flows if
 B is true.

B = false $\rightarrow$ label to which control flows
 if B is false

S.next $\rightarrow$ label of an instruction
 immediately after S.

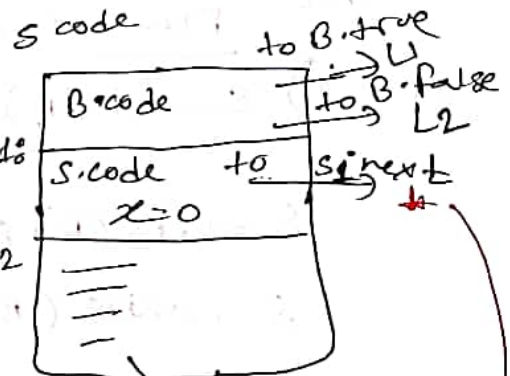
eg) S $\rightarrow$ if (B) S1
 $a + b > c \rightarrow B$ exp $\rightarrow$

S1 $\rightarrow x = 0$
 $\downarrow$
 either L1
 or L2

B = true
 L1
 jao chahi

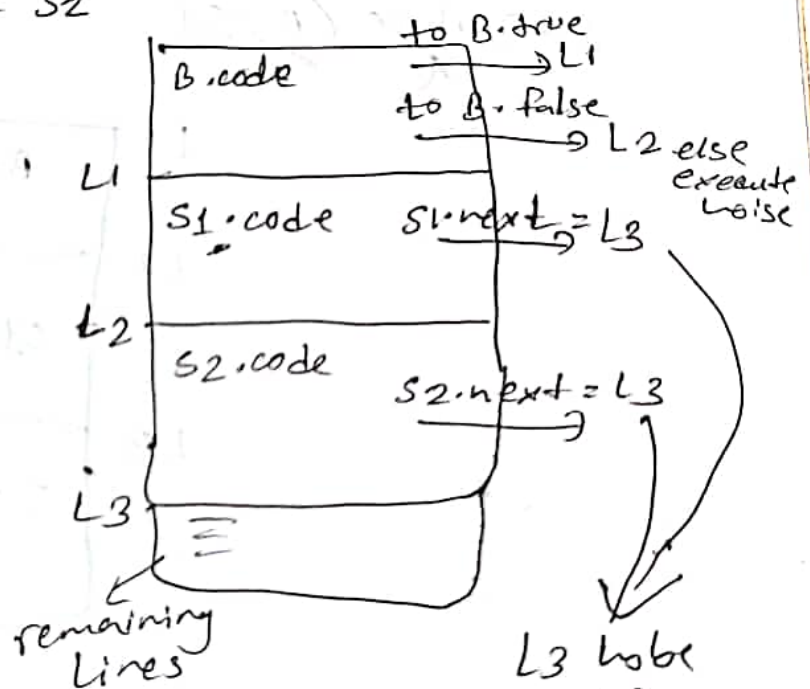
B = false
 normally finish
 execute hobe &
 remaining
 code
 portion.

2
 leave
 hobe
 continue
 korbe



B.code
 B either
 T / F hobe
 T hobe koi
 F hobe koi
 2 F hobe koi
 2 F hobe koi

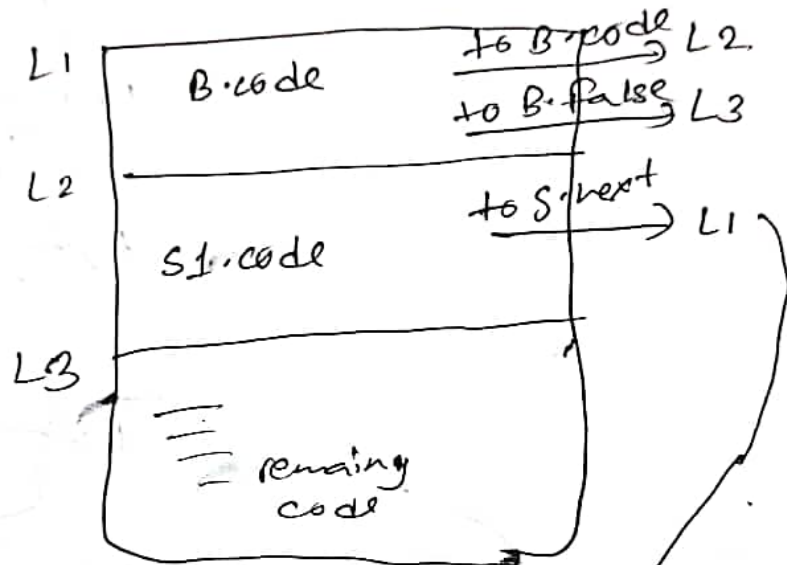
$S \rightarrow \text{if}(B) S_1 \text{ else } S_2$



$L3$ kabe
 ko2
 on no
 condition
 or state-
 ment edhukhe
 -na
 $\therefore L3$ remaining
 lines
 execute
 kabe.

while B hole
 S1 execute karna
 then abar B check karna
 since loop

S. $\rightarrow$ while (B) S1



loop girup
 statement
 execute
 kare
 abar loop
 check
 karte hain

Assignment 4

- $t1 = a - b$
- 1 $t2 = \text{minus } c$
- $t3 = t1 + t2$ ²
- 3 ~~$t4 = t3 * t2$~~
- $z = t4$

quadtuples

	op	arg1	arg2	result
0	-	a	b	t1
1	minus	c		t2
2	+	t1	t2	t3
3	*	t3	t2	t4
4	=	t4		z

triples

	op	arg1	arg2
0	-	a	b
1	minus	c	
2	+	(0)	(1)
3	*	(2)	(1)
4	=	z	(3)

$$a + a * (b - c) + (b - c) * d$$

$$t1 = b - c$$

$$t2 = a * t1$$

$$t3 = a + t2$$

$$t4 = t1 * d$$

$$t5 = t3 + t4$$

b2 c eishob thakena symbol table er jekhane data ache tar pointer er info dei.

constant gula value nishabe thake 2 temp variable gula compiler generated.

i) Assignment instruction:

$x = y \text{ op } z$ [x y z er value operate korchi 2 x er khitore store korchi]
 $\downarrow$
 operator
 2 operands
 $x = y \text{ op } z$ (normal $\rightarrow$)

ii) Assignment instructions for unary operators:

$x = \text{op } y$ [Only 1 operand, $x = -y$ unary minus]
 1 operand
 = logical negation
 = shift operation
 = conversion (y k int/string e convert korchi)

iii) copy instruction:

$x = y$ [x y er val x e store korchi]

iv) Unconditional jump

jekono line no. diye bolte part oikhane jay jao / diye bolte pari go to L

goto L.

L: $\equiv$ } L er jonno/nake code likha ache jekha este eishob code execute korbo.

v) Conditional jump:

true hole ekta jaygay jabe 2 False hole arekta jaygay
 if x goto L
 if false goto L

vi) conditional jump

v) conditional jump for relational operators.

if (a > b) ~~if~~ if x relop y goto L
 el-to satisfy hole L e jabe.

vii) for functions

sum(x1, x2, x3)
 ↓
 func name
 3 param niche
 identify func name,
 param koita param
 niche
 e ki ki param
 niche

param 0 x1
 param x2
 param x3
 call for sum, 3 no. of param.
 ↓
 func name

viii) Index copy instructions

$x = y[i] / x[i] = y$

[y er jai base add ache,
 y er ita
 index er val
 ta se e
 store korch]

y [] [] []
 base = 200

ix) Address & pointer assignment

$x = &y$ → [y er add = x er actual value]

$x = *y$ → [y is a pointer jai info x e shift korch]
 → [y is a pointer whose info is being shifted to x]

$*x = y$ → [x jai variable k point kore ache sheita = y]
 → the variable which is being pointed by $x = y$

Array base add calc

y =

0	1	2	3	4	5
---	---	---	---	---	---

 200 base add

len = 6
 base add is y er starting
 index

base add of y = 200

Ind hole 4 byte kore mem allocate korch
 float " 8 " " " "

1st index
 er val fetch
 kore korch
 chaita 204
 x mem add
 e kore jabo.

1st index	204	205	206	207
0th "	200	201	202	203

3rd index $200 + 4(3) = 212$ 2012 * mem add e jabo.
 er val pete

$200 + (i * 4) = \text{memory add for 1 dimensional array.}$
 ↓
 base add ↓ jete index e jete chal ↳ koto byte kore mem allocate kori

Quadruples representation

Triples representation

quadruples instruction no.

3 add code

quadruples representation of this 3 add code

↓	op	1st arg	2nd arg	result
0	uminus	c		t1
1	*	b	t1	t2
2	uminus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a
	op	arg1	arg2	result

$a = b * (-c) + b * (-c)$

unary operator minus

$a = b * -c + b * -c$

$t1 = -c$

$t1 = \text{uminus } c$

$t2 = b * t1$

$t3 = \text{uminus } c$

$t4 = b * t3$

$t5 = t2 + t4$

$a = t5$

copy instruction only assign to hoche

Ha is the result

#t5 is the argument

- quadruples er khetre condition/uncondition jump thake, jekha jump korbo oita? result go to f.

delta result e boshbe.

param x1

x u x 2

era arg 2

2 result column use korena

1st arg → first argument of the operator.
 operator e left e k ache

2nd argument → 2nd of the operation
 operator e right e k ache.

$t1 = \text{uminus } c$ [c k unary minus kor
 operator 2 store int t1
 1 argument only c.]

3 add code quadruples e lagbe.

quadruples er jonno 3 add code lagbe

// syntax tree

dileo
 3 add code e convert
 kora lagbe

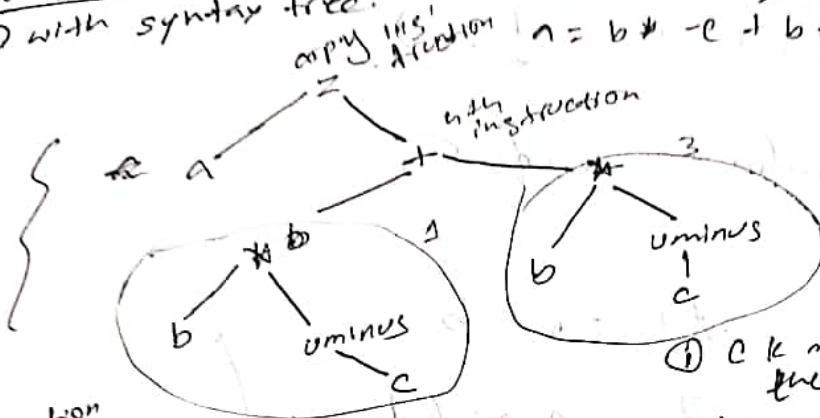
cor quadruples e result column ache jekha assign kora gay.

Triples

→ with syntax tree.

$$a = b * c + b * -c$$

$$a = b * -c + b * -c$$



no result column

Instruction no.

Instruction no.	op	arg 1	arg 2
0	uminus	c	
1	*	b	(0)
2	uminus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

① c k minus korchhi then b k shantre multiply.

② c k first e minus kori

③ then b c je c minus korlam oitake b-c r shantre multiply korbo

$b * -c$
since result hai, we use instruction no.

→ copy kore
arg 1 e kar bhetore assign karbo
arg 2 kar val ta arg 1 e assign korchilam.

Indirect triples

→ triple representation thakbe + ekta extra table thakbe jeta table e ins no. gular shantre ekta pointer info thakbe.

pointer info	ins no.
35	0
36	1
37	2
38	3
39	4
40	5